# NETWORK ADMINISTRATOR PRACTICAL ASSIGNMENT

## Linux Command Practice and an AI-Powered Diagnostic Agent

**Student:** Ameed Qadah
**Student ID:** 12401336
**Environment:** Ubuntu 64-bit on VMware Workstation
**Practice date:** 12 October 2026

Submission evidence: 38 screenshots, Python AI agent source, and generated diagnostic log.

## 1. Assignment Objectives

Practice the Linux commands from Lecture 3, understand their behavior, capture terminal evidence, and build an AI agent that autonomously selects and executes permitted commands within the Ubuntu virtual machine.

## 2. Command Reference

| # | Command | Purpose |
|---|---|---|
| 1 | `man ls` | Displays the ls manual page. |
| 2 | `man -k intro` | Finds manual entries matching intro in names/descriptions. |
| 3 | `man -K intro` | Searches full manual-page content for intro. |
| 4 | `login` | Attempts to start a login session; this terminal returned Hangup. |
| 5 | `passwd` | Changes the account password; execution was documented. |
| 6 | `logout` | Ends the login shell and returns to the parent shell. |
| 7 | `exit` | Closes the current shell. |
| 8 | `pwd` | Prints the current working directory. |
| 9 | `cd /home` | Moves to the /home directory. |
| 10 | `ls -la` | Lists all entries, including hidden ones, in long format. |
| 11 | `mkdir testdir` | Creates the testdir directory. |
| 12 | `cp file1.txt file2.txt` | Copies a file, replacing the destination content when applicable. |
| 13 | `cp -r dir1 dir2` | Recursively copies a directory. |
| 14 | `mv file1.txt file2.txt` | Moves or renames a file. |
| 15 | `rm file1.txt` | Removes the specified file. |
| 16 | `rmdir empty_dir` | Removes an empty directory. |
| 17 | `rm -r dir1` | Recursively removes the specified directory. |
| 18 | `cat file1.txt` | Displays the file contents. |

| # | Command | Purpose |
|---|---|---|
| 37 | `more file1.txt` | Pages through a text file. |
| 38 | `less file1.txt` | Interactively navigates a text file. |
| 39 | `head -5 file1.txt` | Displays up to the first five lines. |
| 40 | `tail -5 file1.txt` | Displays up to the last five lines. |
| 41 | `wc file1.txt` | Counts lines, words, and bytes. |
| 42 | `sort file1.txt` | Sorts lines in ascending order. |
| 43 | `sort -r file1.txt` | Sorts lines in reverse order. |
| 44 | `sort -t: -n -k 3 file1.txt` | Numerically sorts by the third colon-separated field. |
| 45 | `uniq file1.txt` | Filters adjacent duplicate lines. |
| 46 | `cut -f1 -d: file1.txt` | Extracts the first colon-delimited field. |
| 47 | `grep "search_term" file1.txt` | Prints lines matching the specified pattern. |
| 48 | `paste file1.txt file2.txt` | Combines corresponding lines side by side. |
| 49 | `tr 'a' 'b' < file1.txt` | Translates lowercase a to b in the displayed stream. |
| 50 | `date` | Displays system date and time. |
| 51 | `cal` | Displays a monthly calendar. |
| 52 | `echo "Hello World"` | Prints the specified text. |
| 53 | `vi filename` | Opens a file in the vi editor for editing. |

Note: Destructive file commands were exercised in a dedicated practice folder. Login sessions and password changes were tested on the VM. Some screenshots document multiple commands together.

## 3. AI Network Administrator Agent

A locally hosted Ollama model (qwen2.5:0.5b) was connected to a Python program. The program prompts the model to choose diagnostic actions from a validated allowlist. Each action runs using subprocess without shell=True, returns output to the program, and is analyzed by the model.
**Implemented safeguards:** fixed command allowlist; command-selection validation; execution timeout; no free-form shell execution; limited output sent to the model.
**Allowed diagnostic categories:** network addresses, routing, ping, DNS lookup, listening ports, hostname, memory, disk and system information.

## 4. Verified Agent Trial

| Item | Observed result |
|---|---|
| Chosen commands | ip -brief address; ip route; ping -c 2 -W 2 1.1.1.1 |
| Interface / VM address | ens33 / 192.168.159.128/24 |
| Default gateway | 192.168.159.2 |
| Ping | 2 packets received of 2 transmitted; 0% packet loss |
| Average round-trip time | 50.670 ms |
| Generated report | report_20261012_223415.txt |

## 5. Limitations and Troubleshooting

An early AI analysis request timed out after 180 seconds. Reducing model context and generation limits allowed a later run to finish and save a report.
The small model also misidentified the VM address as the gateway in its generated prose. The raw command output is authoritative: 192.168.159.128 is the VM address, and 192.168.159.2 is the gateway. The AI did not execute DNS or listening-port checks in the verified run; these checks remain available but were not demonstrated in that run.

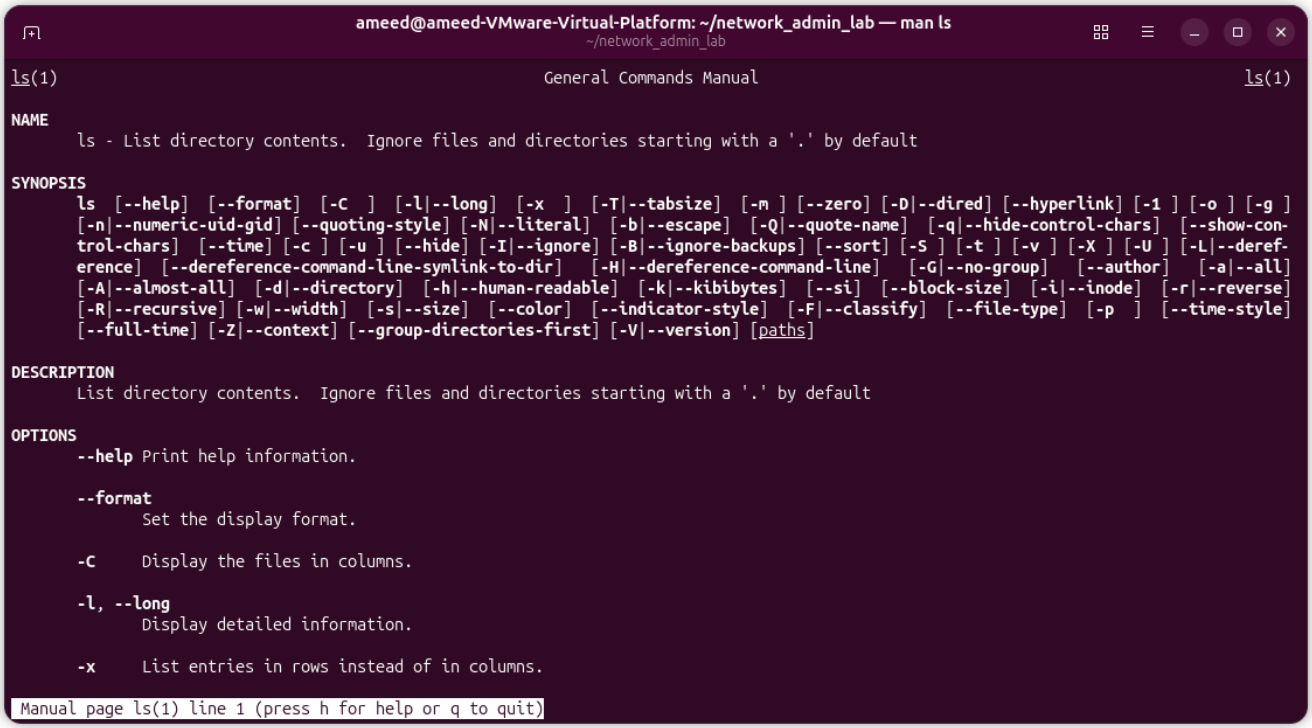
## 6. Conclusion

The exercise demonstrates Linux command-line administration, local AI planning, automatic execution of bounded diagnostic commands, interpretation of command output, and report generation. Human review remains necessary for model-generated explanations.

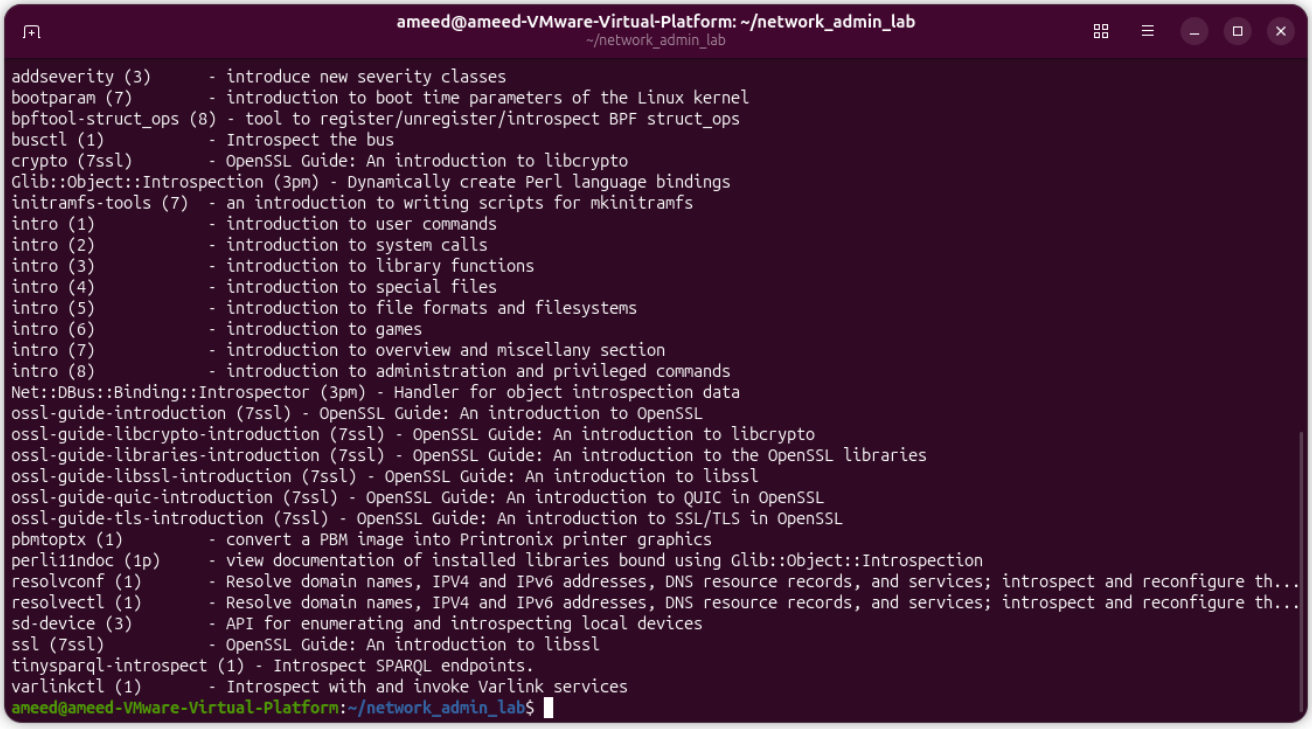
# Appendix A - Screenshot Evidence

The following pages reproduce all 38 screenshots supplied in the assignment archive, in chronological order. Each retains its original timestamp filename so evidence can be traced back to the VM capture.

**Evidence 01** - 21-45-38 / man ls



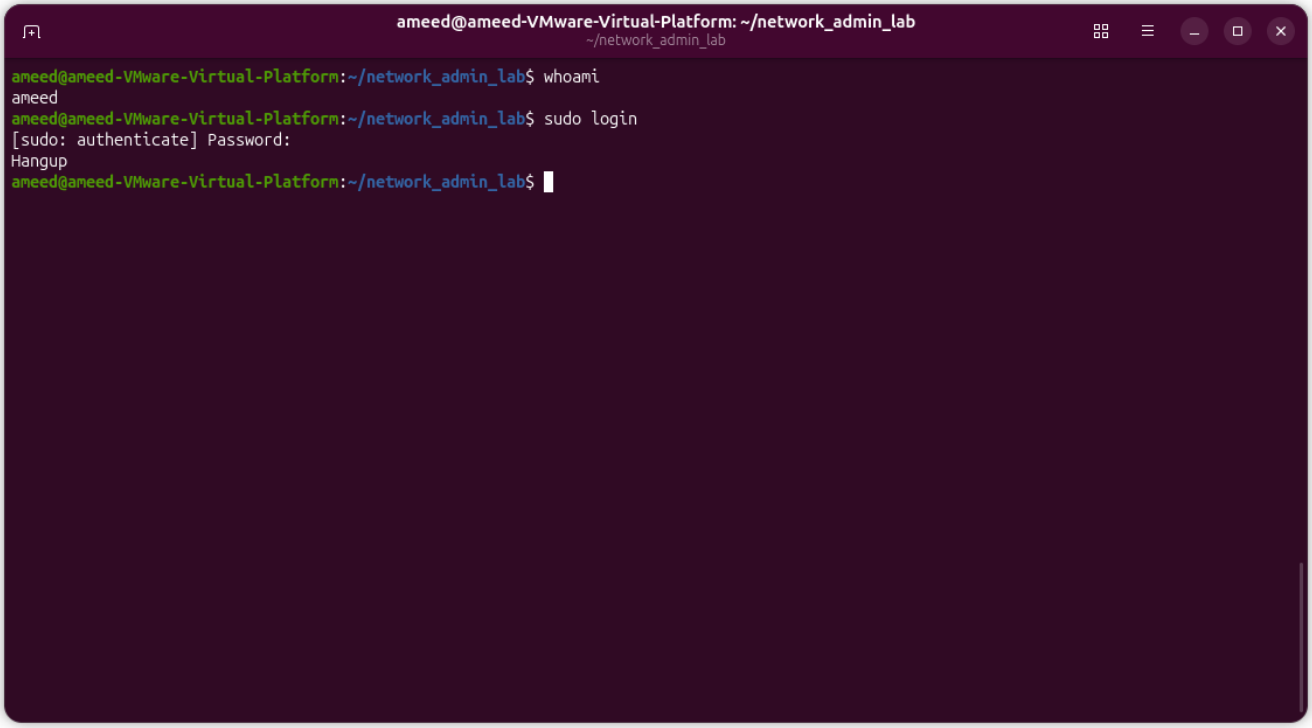

**Evidence 02** - 21-46-15 / man -k intro



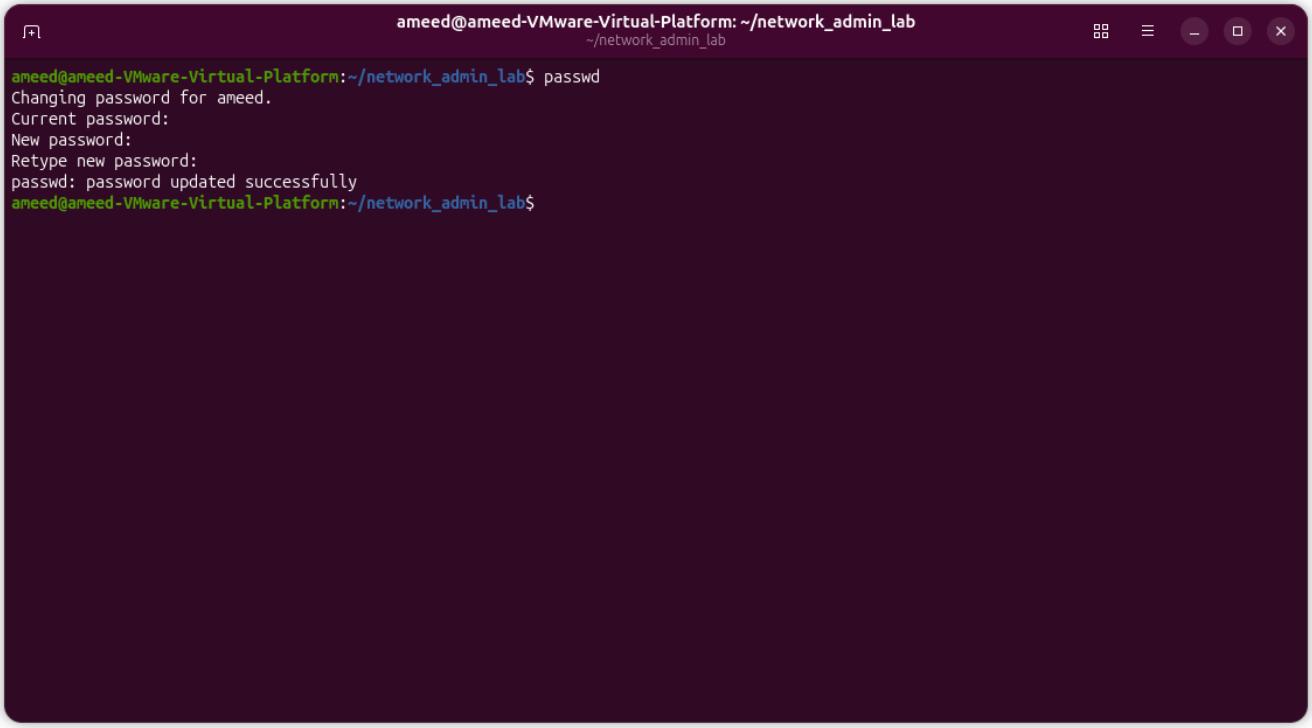
**Evidence 03** - 21-47-34 / man -K intro



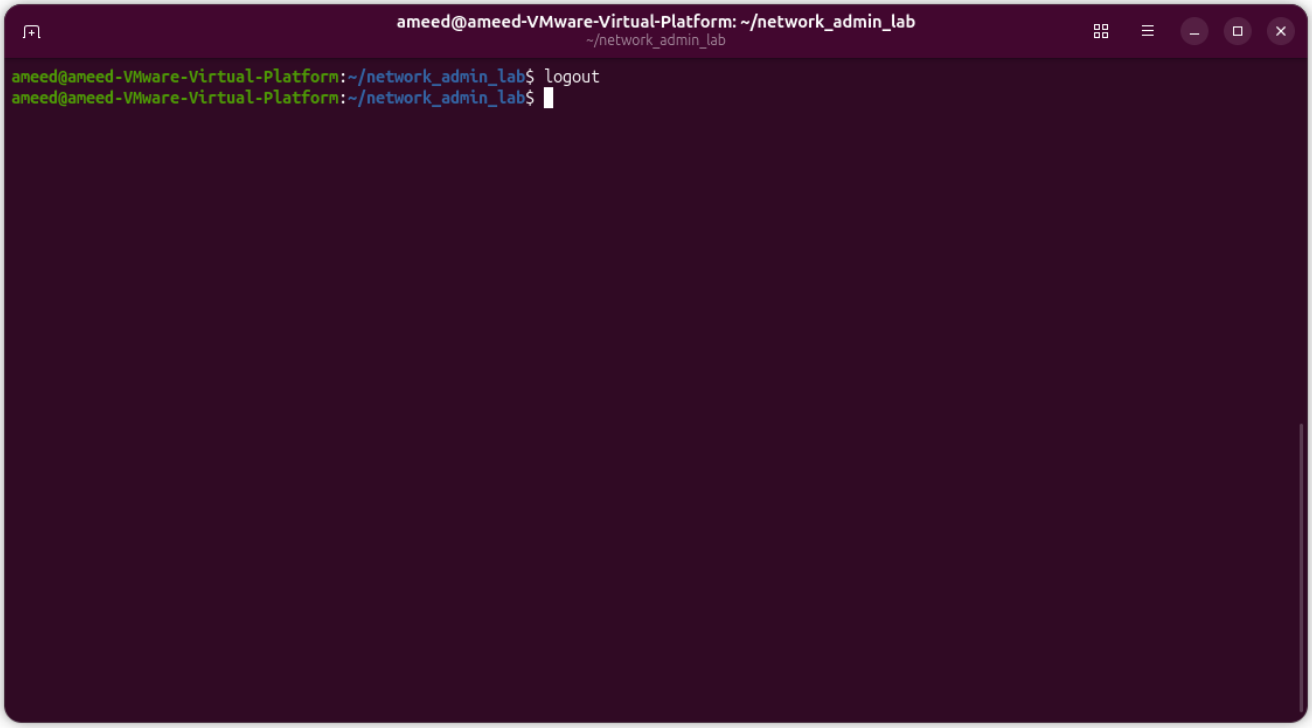

**Evidence 04** - 21-48-40 / login attempt



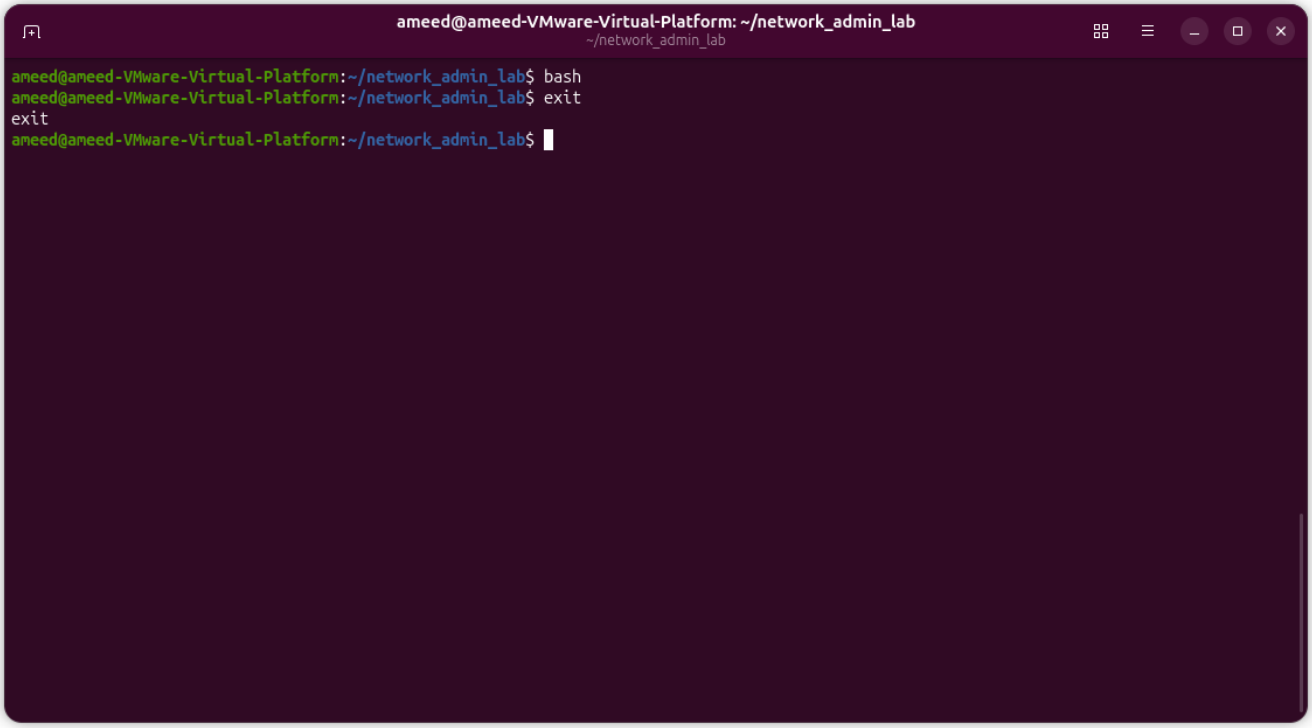
**Evidence 05** - 21-51-45 / passwd



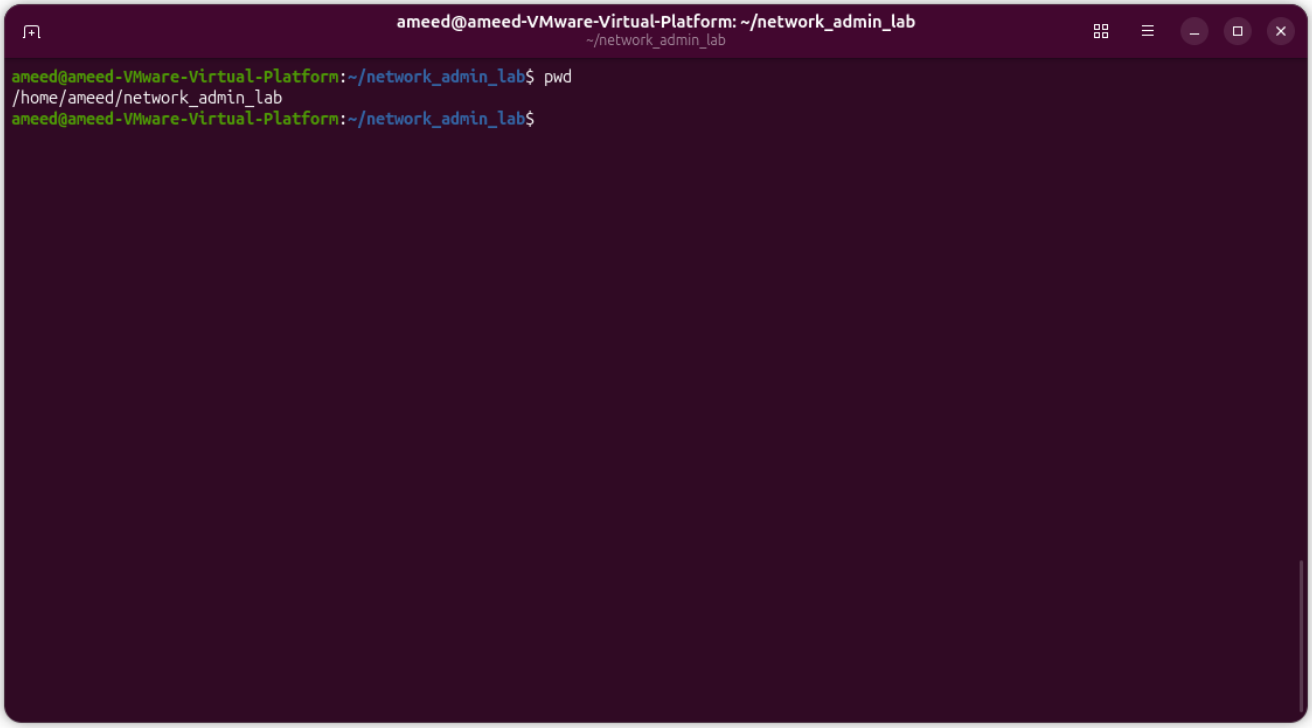

**Evidence 06** - 21-55-08 / logout



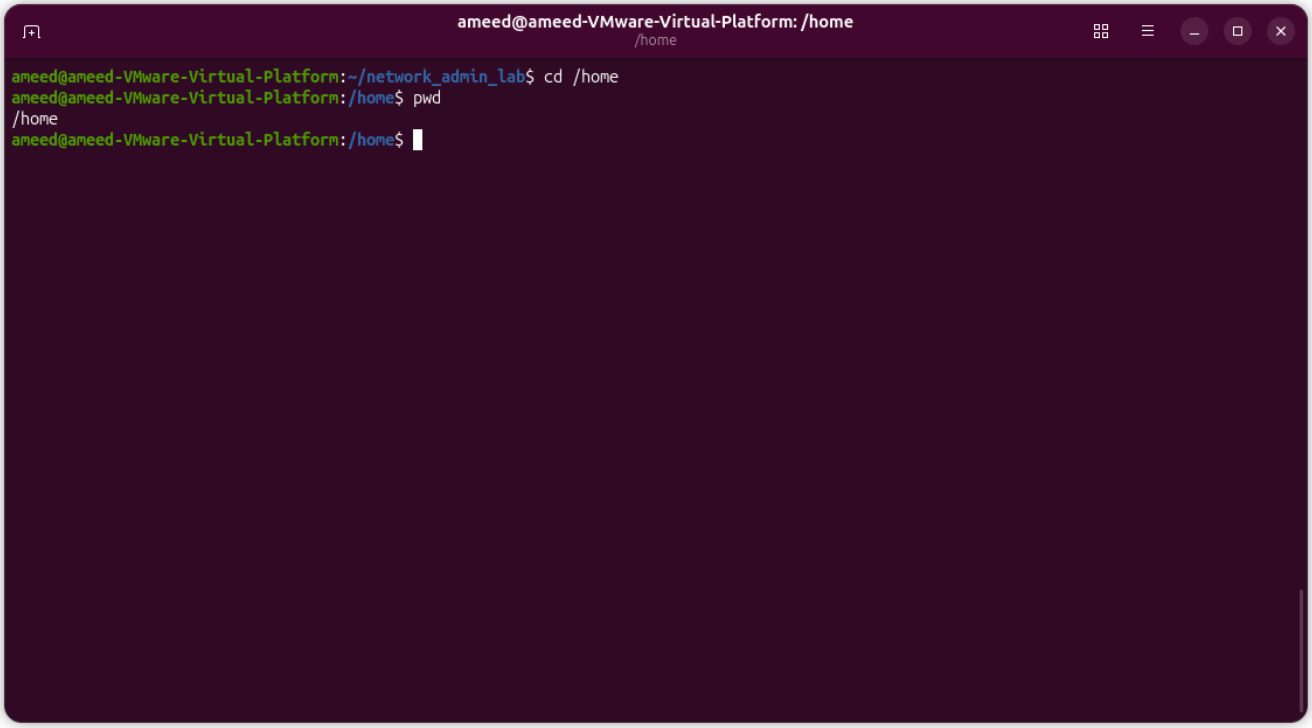
**Evidence 07** - 21-55-54 / exit



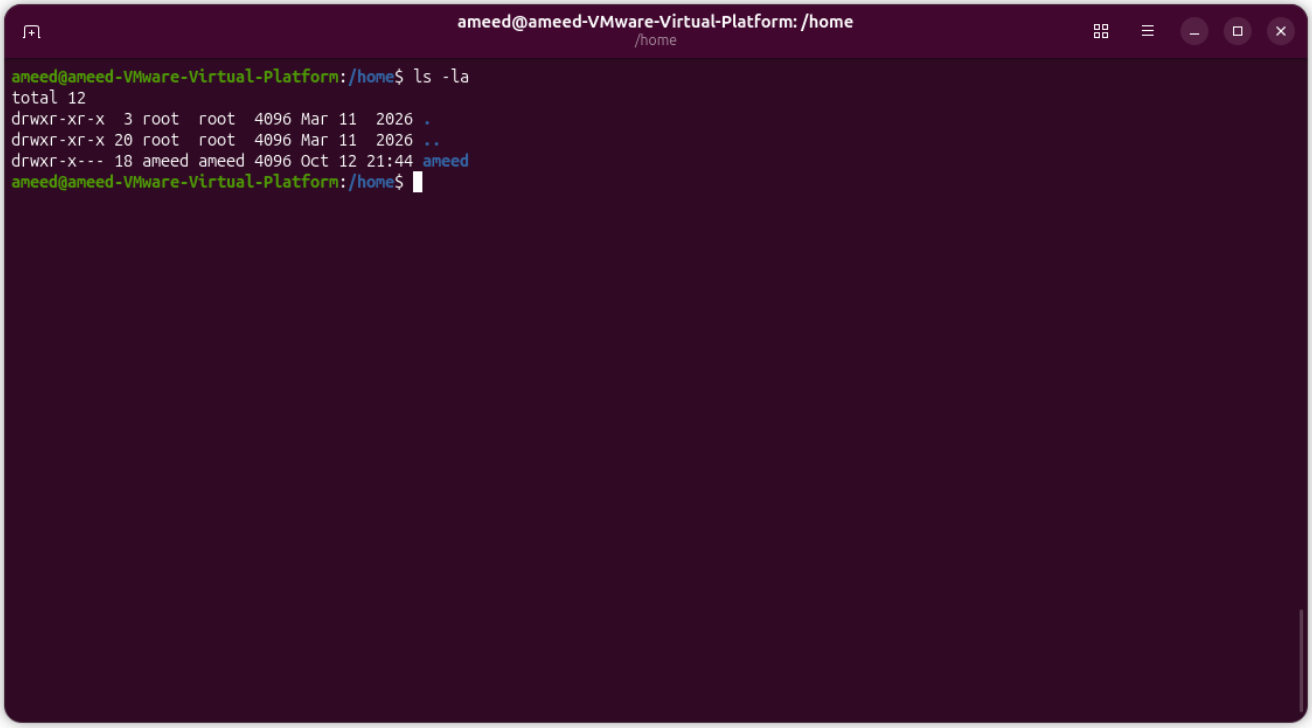

**Evidence 08** - 21-56-10 / pwd



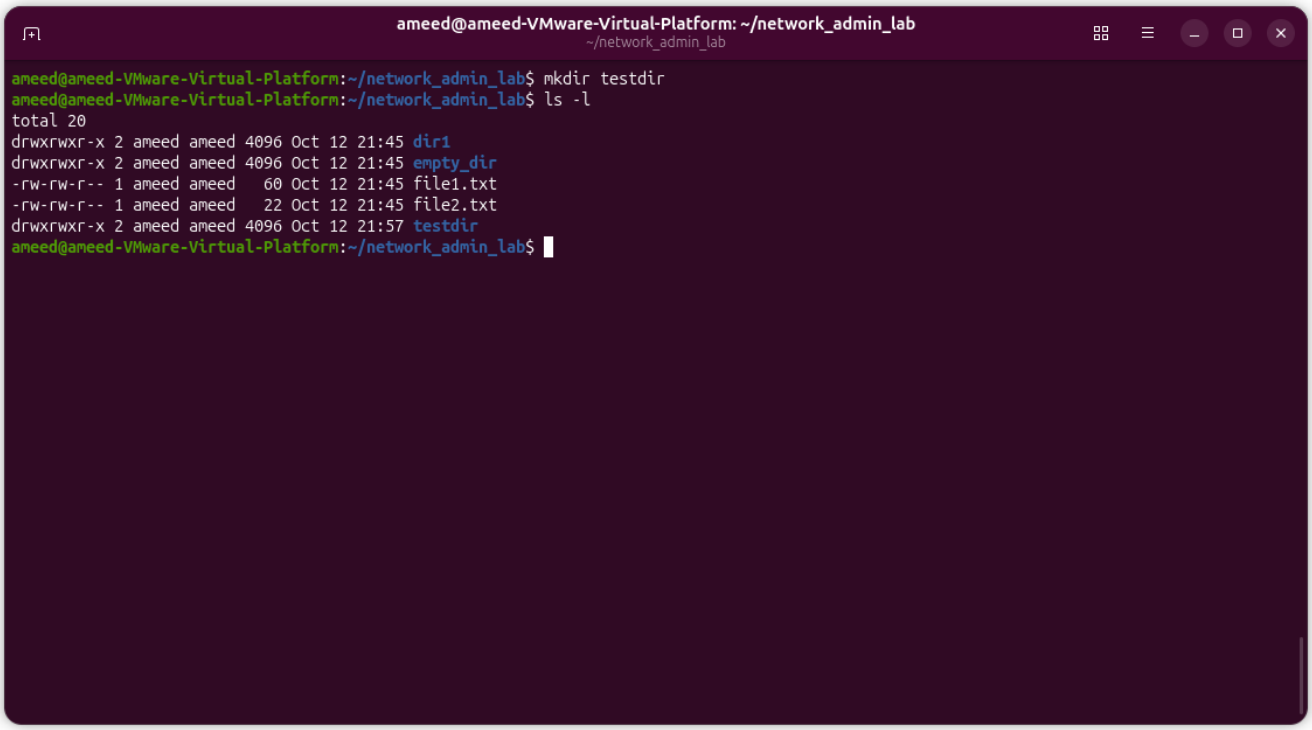
**Evidence 09** - 21-56-39 / cd /home



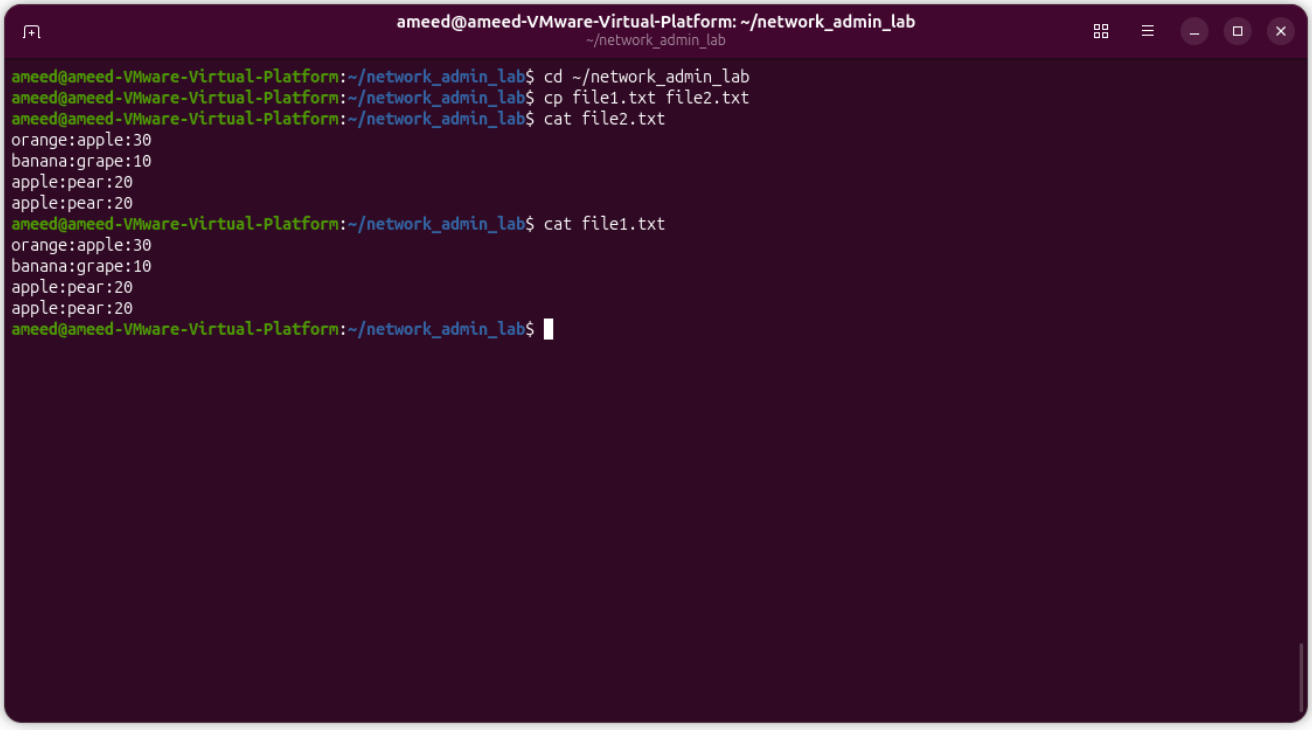

**Evidence 10** - 21-57-09 / ls -la



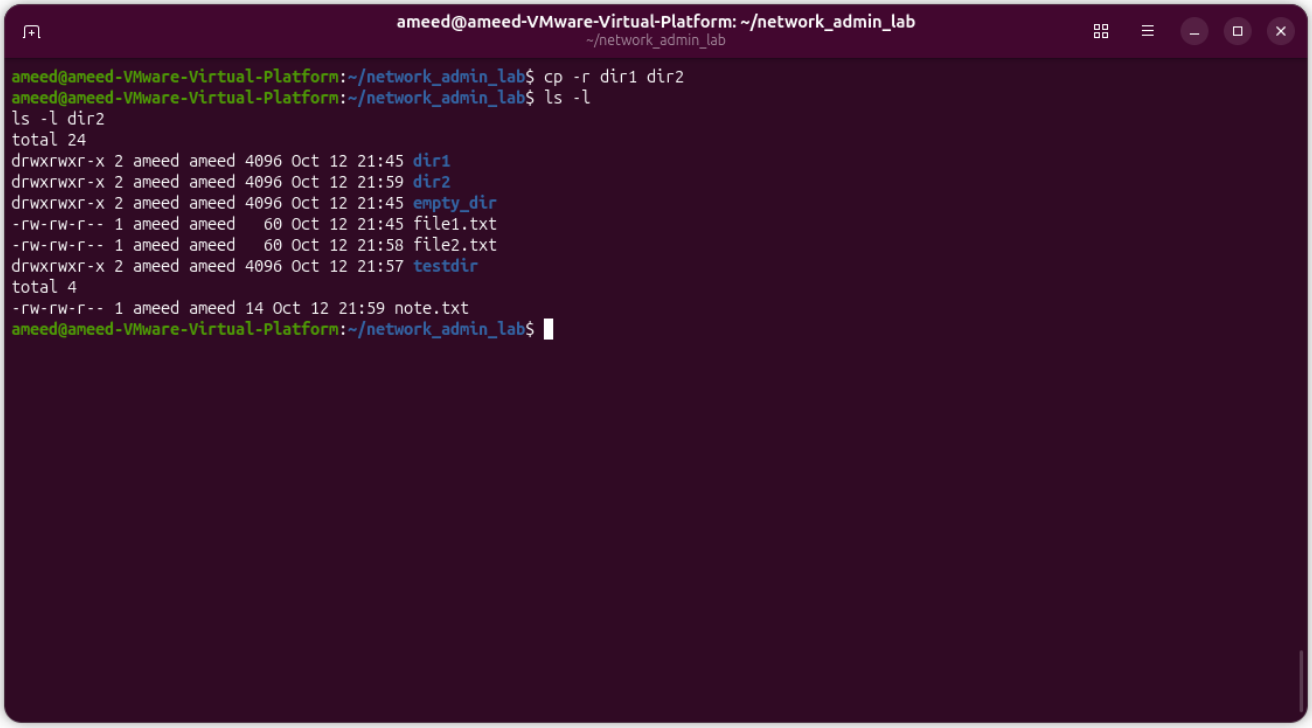
**Evidence 11** - 21-57-58 / mkdir testdir



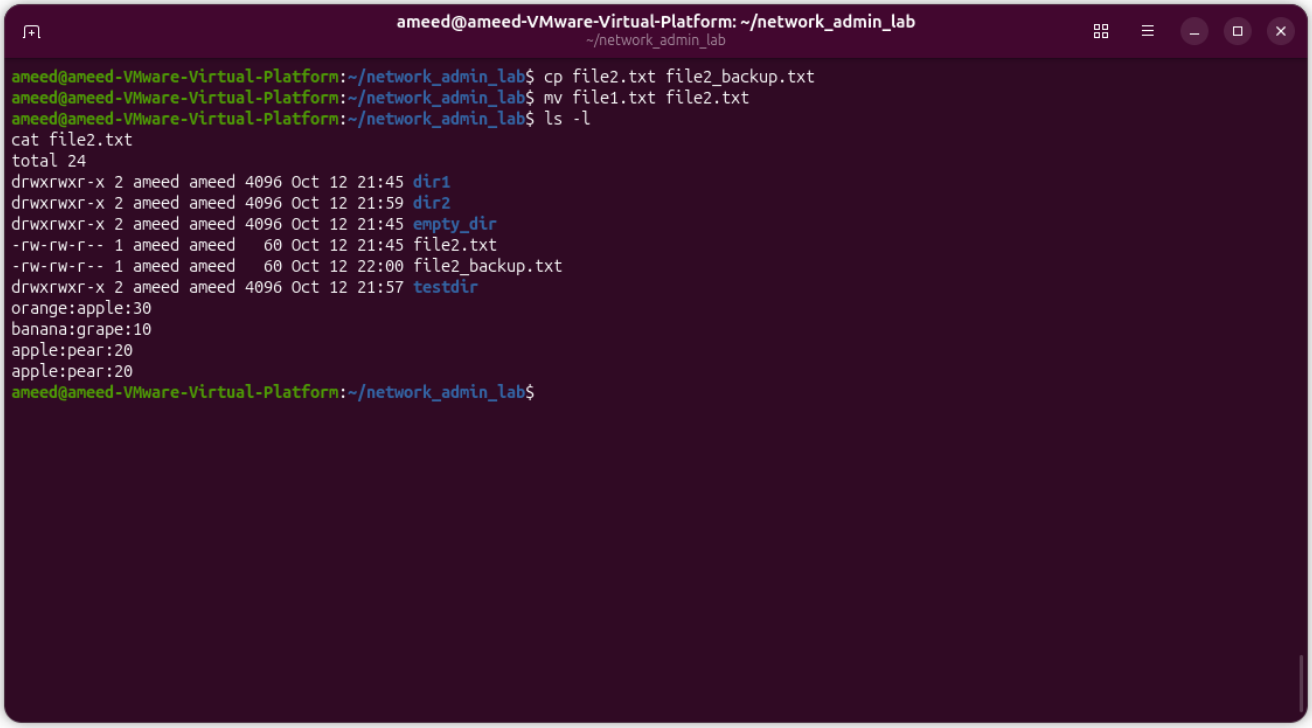

**Evidence 12** - 21-58-49 / cp file1.txt file2.txt



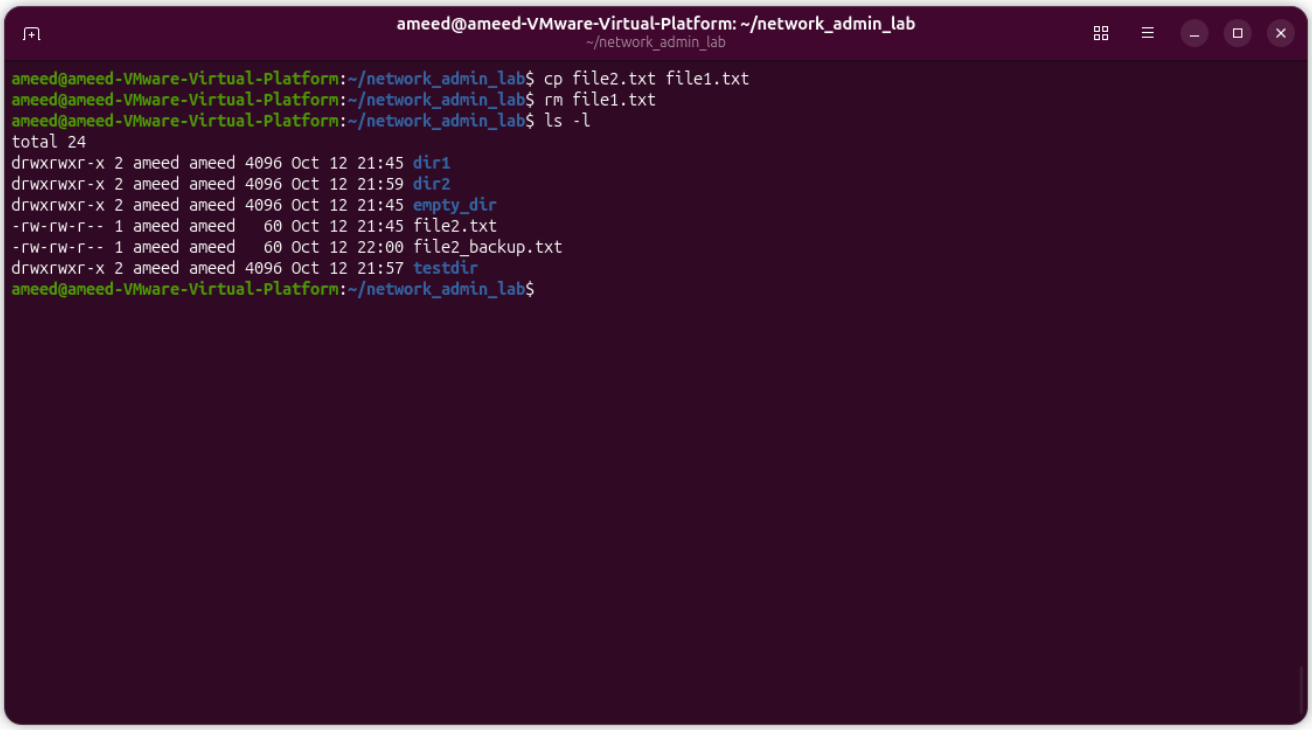
**Evidence 13** - 21-59-48 / cp -r dir1 dir2



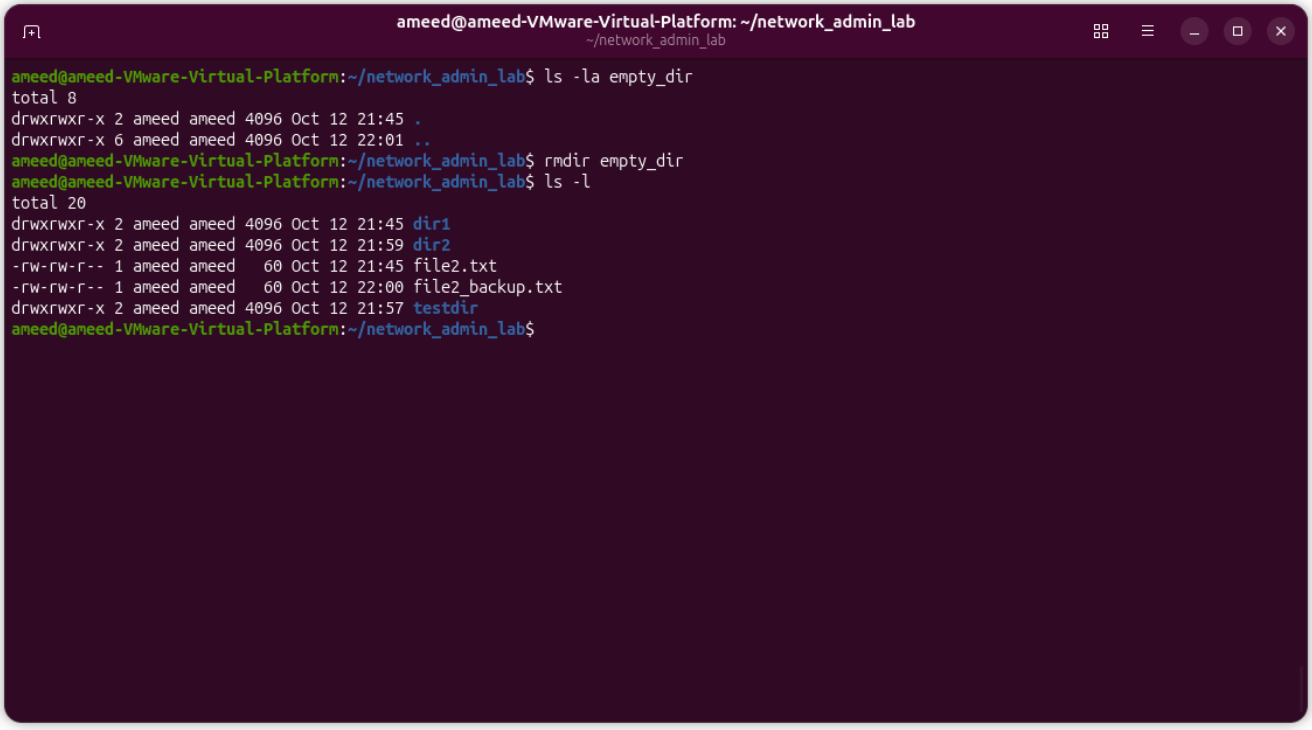

**Evidence 14** - 22-00-33 / mv file1.txt file2.txt



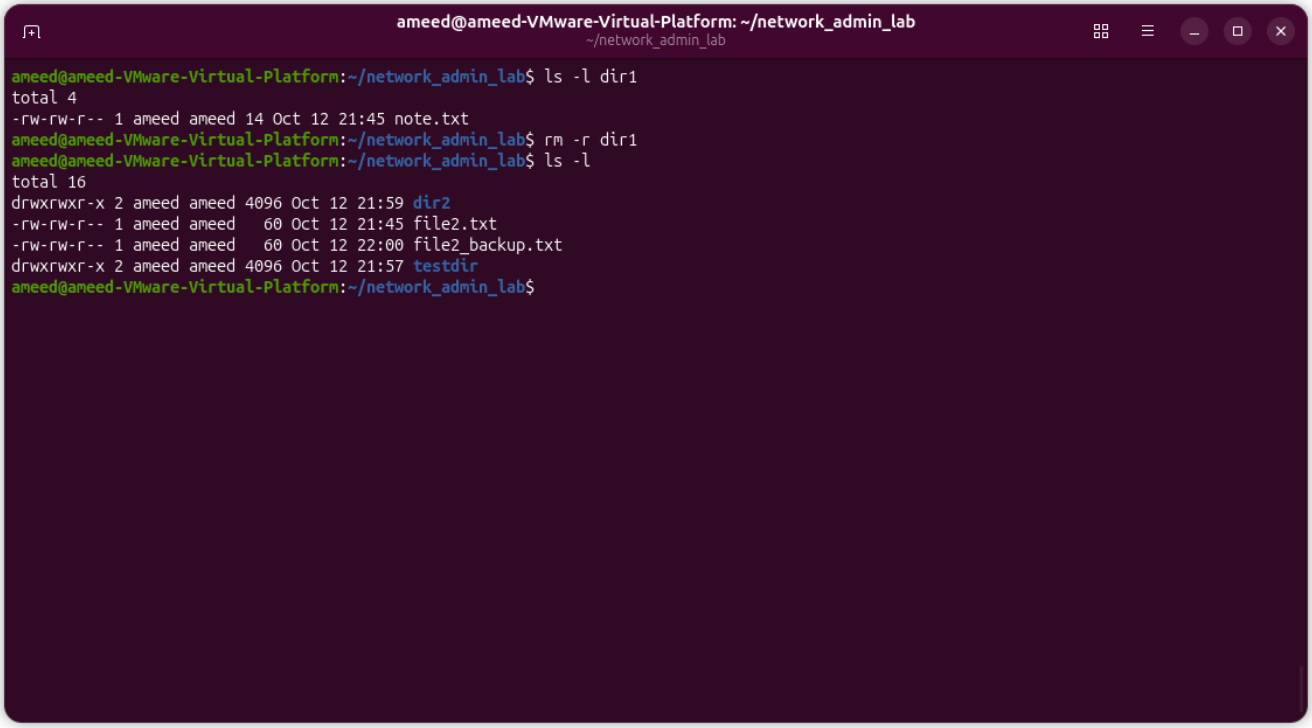
**Evidence 15** - 22-01-18 / rm file1.txt



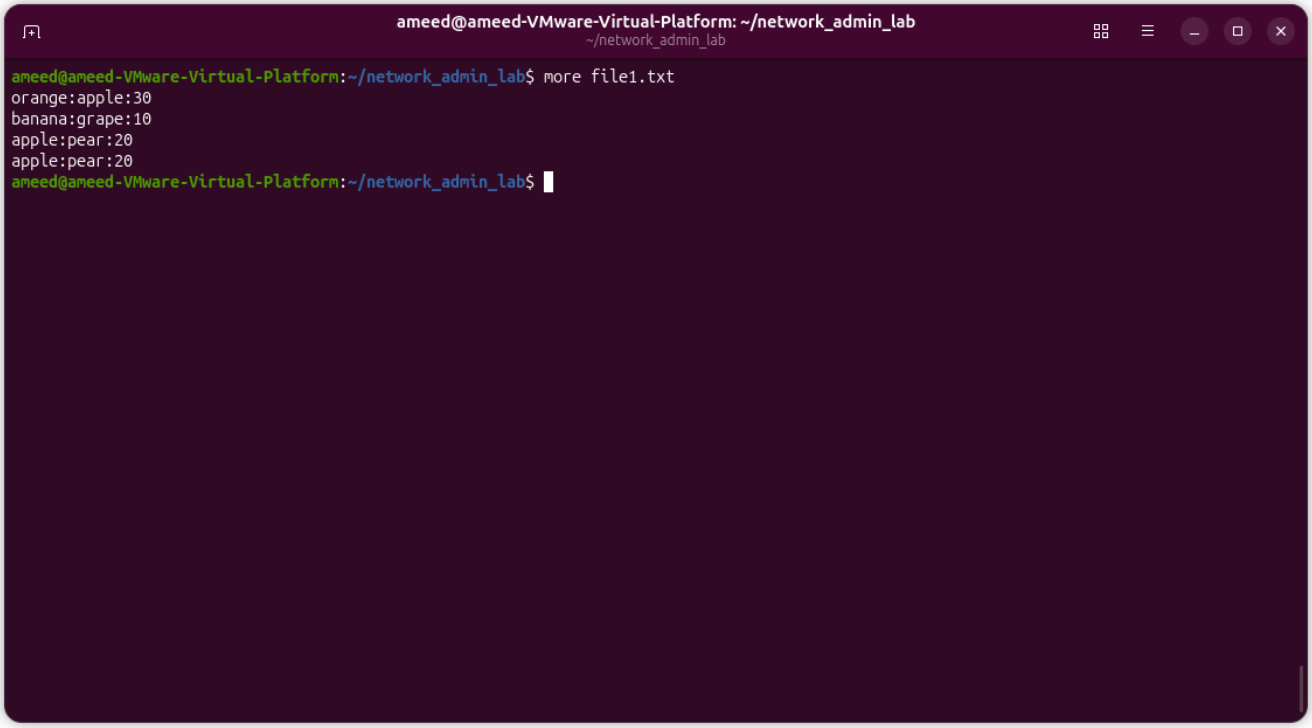

**Evidence 16** - 22-02-07 / rmdir empty_dir



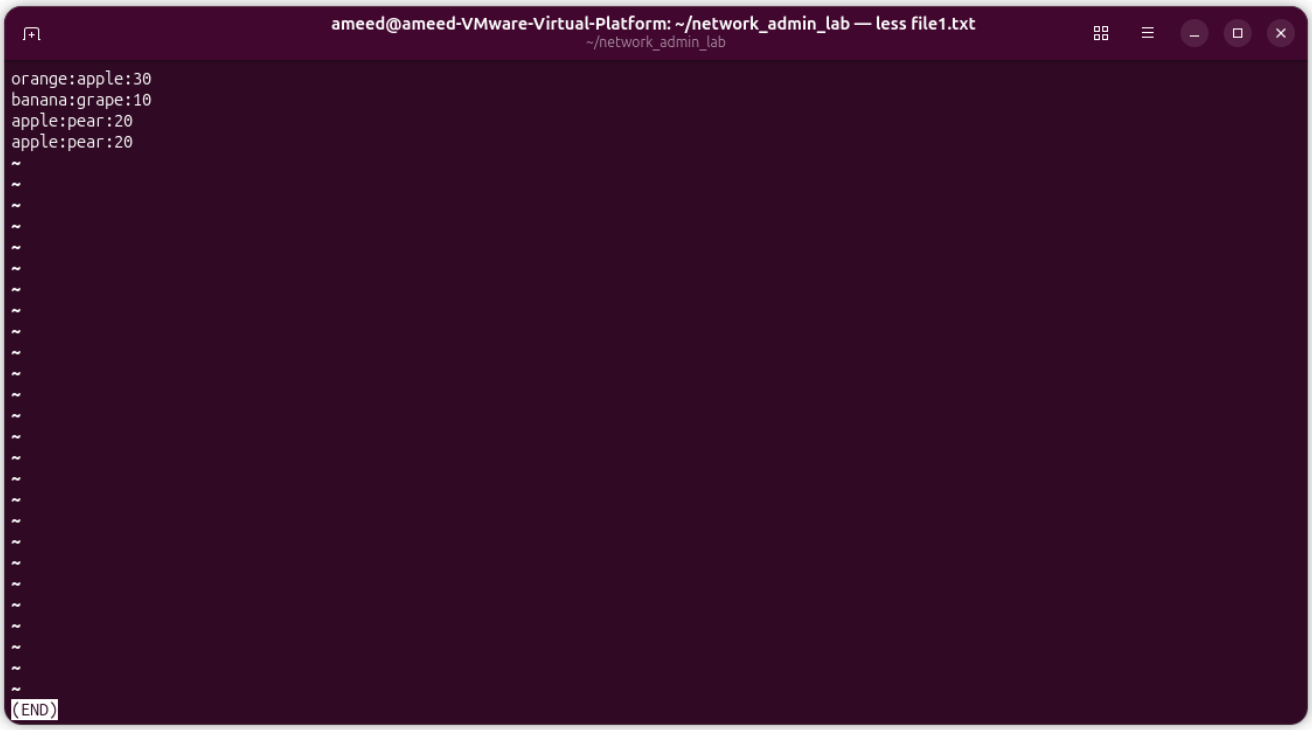
**Evidence 17** - 22-02-48 / rm -r dir1



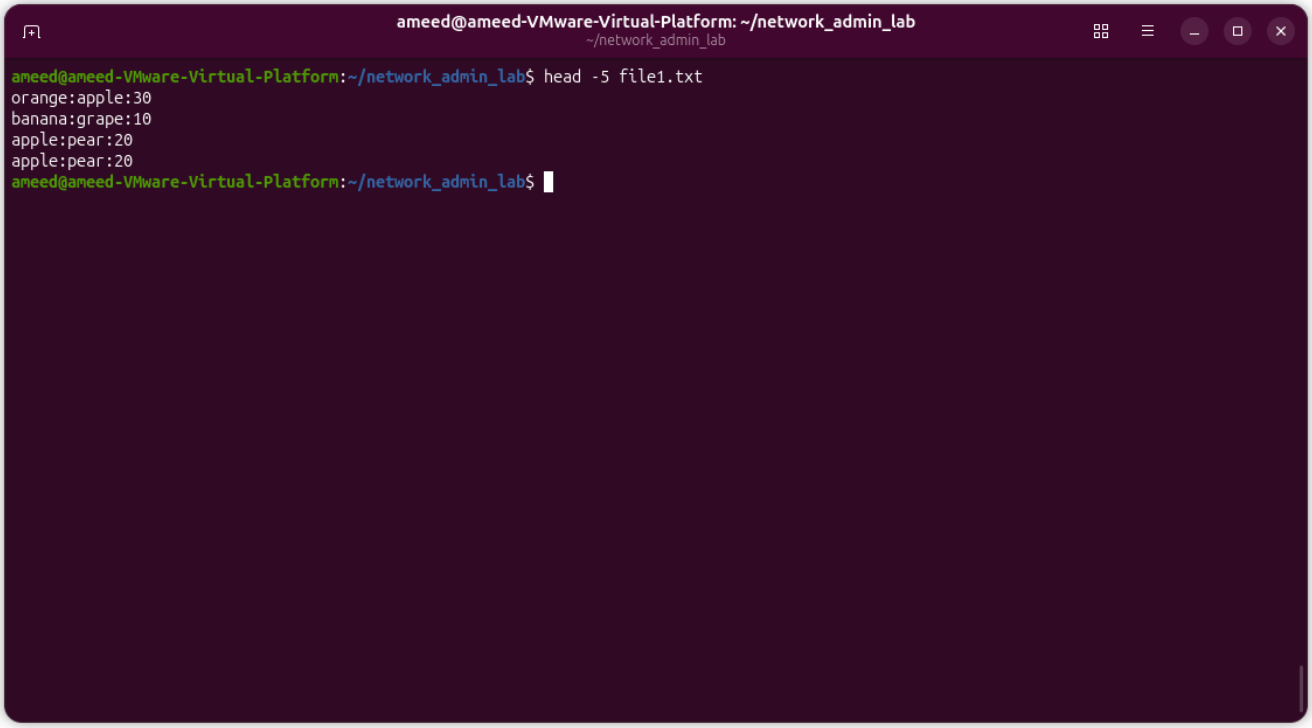

**Evidence 18** - 22-04-15 / cat file1.txt



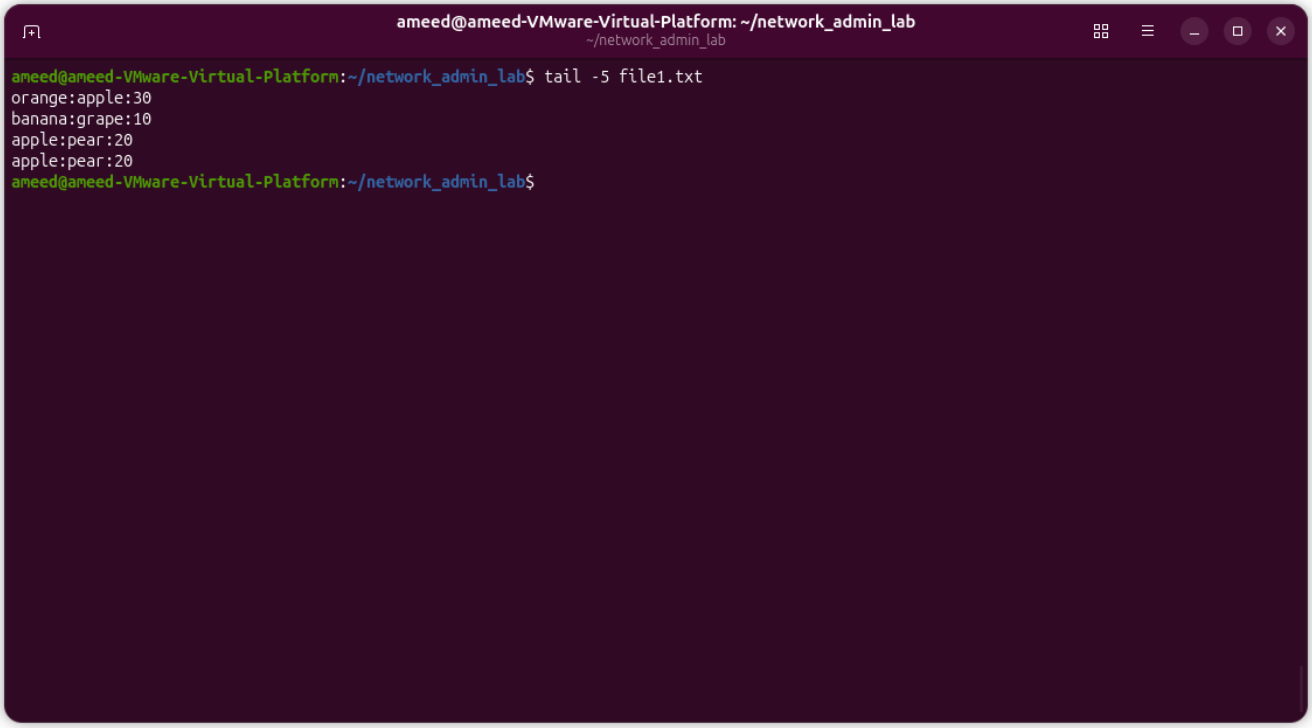
**Evidence 19** - 22-04-56 / less file1.txt



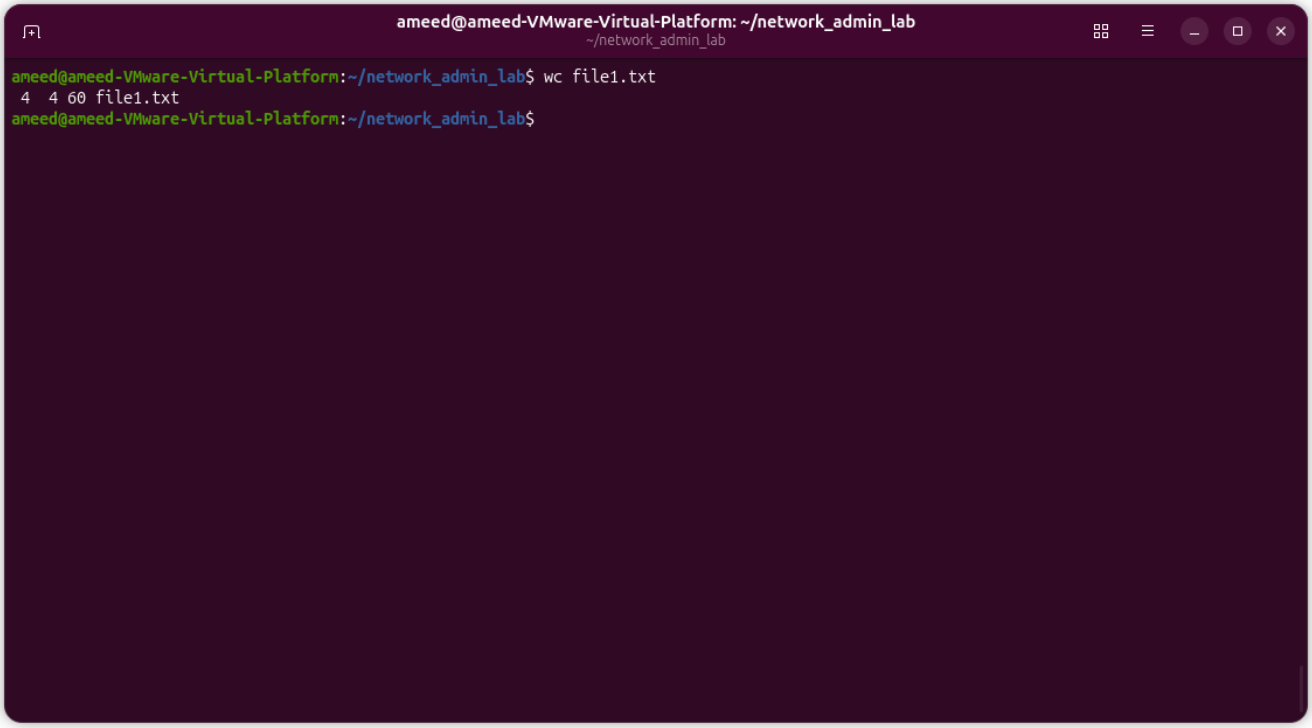

**Evidence 20** - 22-05-31 / more / head sequence



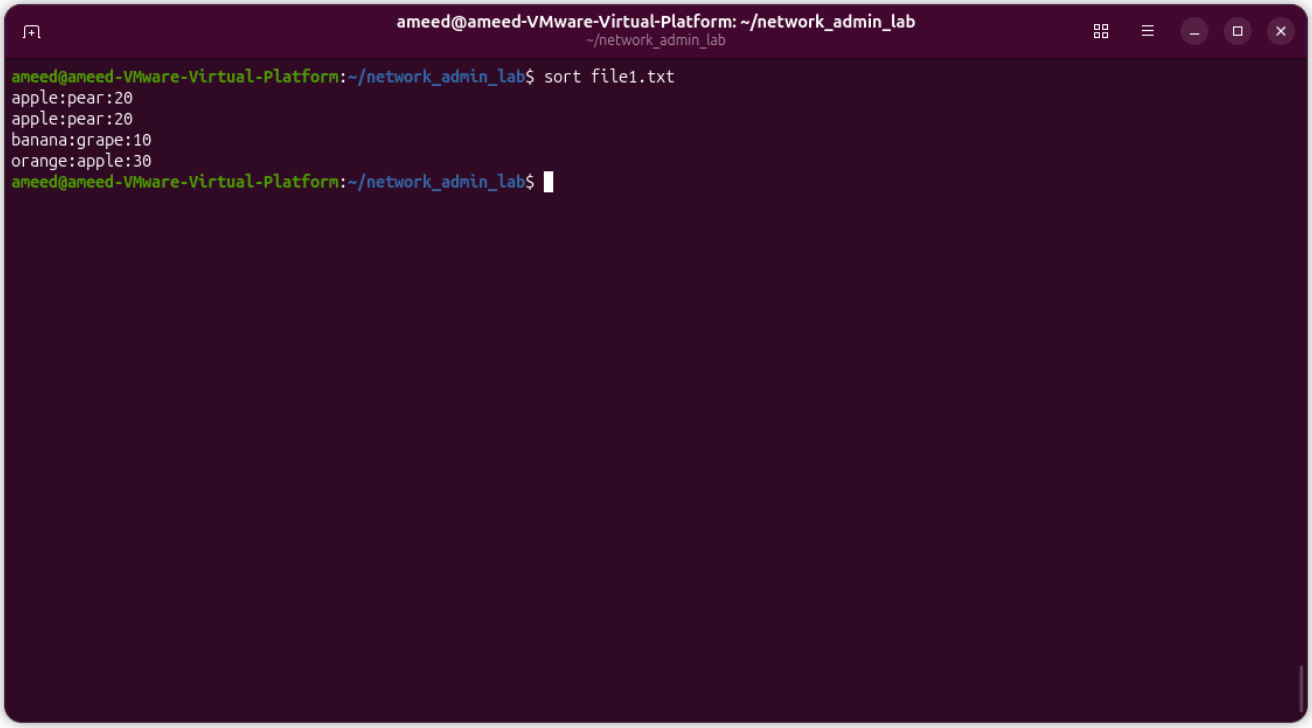
**Evidence 21** - 22-05-51 / tail -5 file1.txt



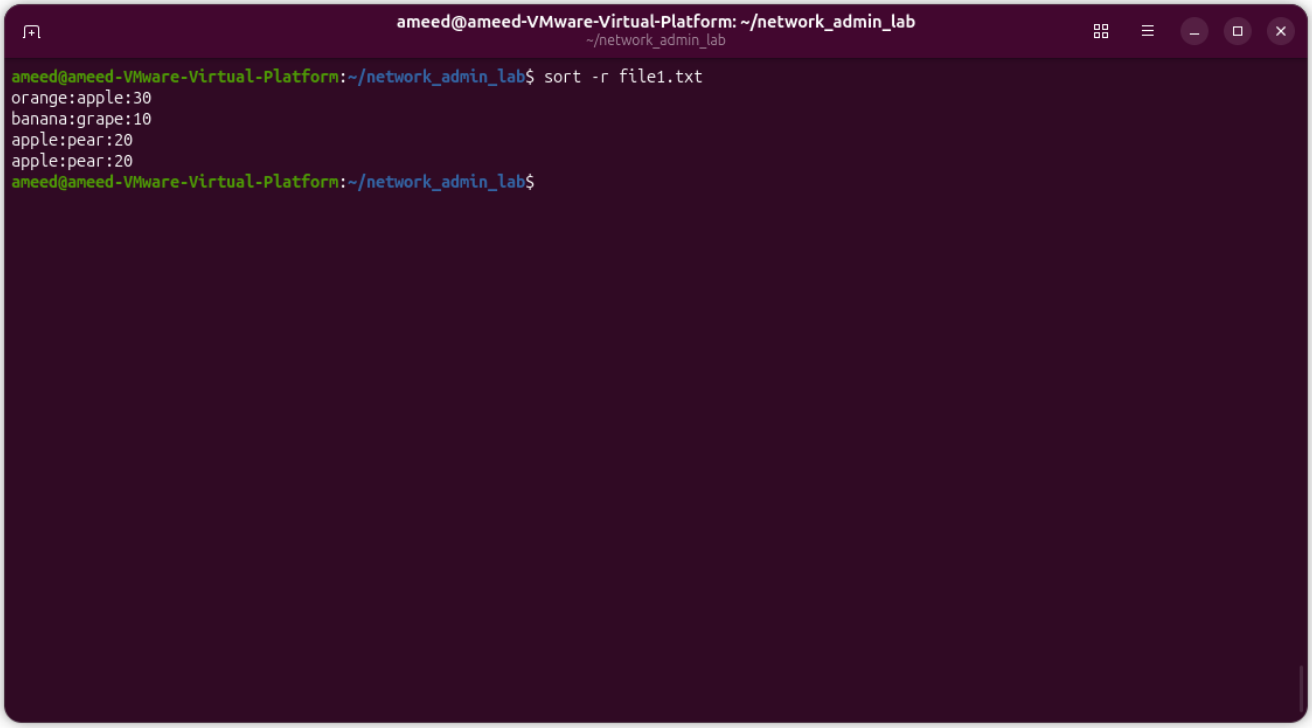

**Evidence 22** - 22-06-12 / wc file1.txt



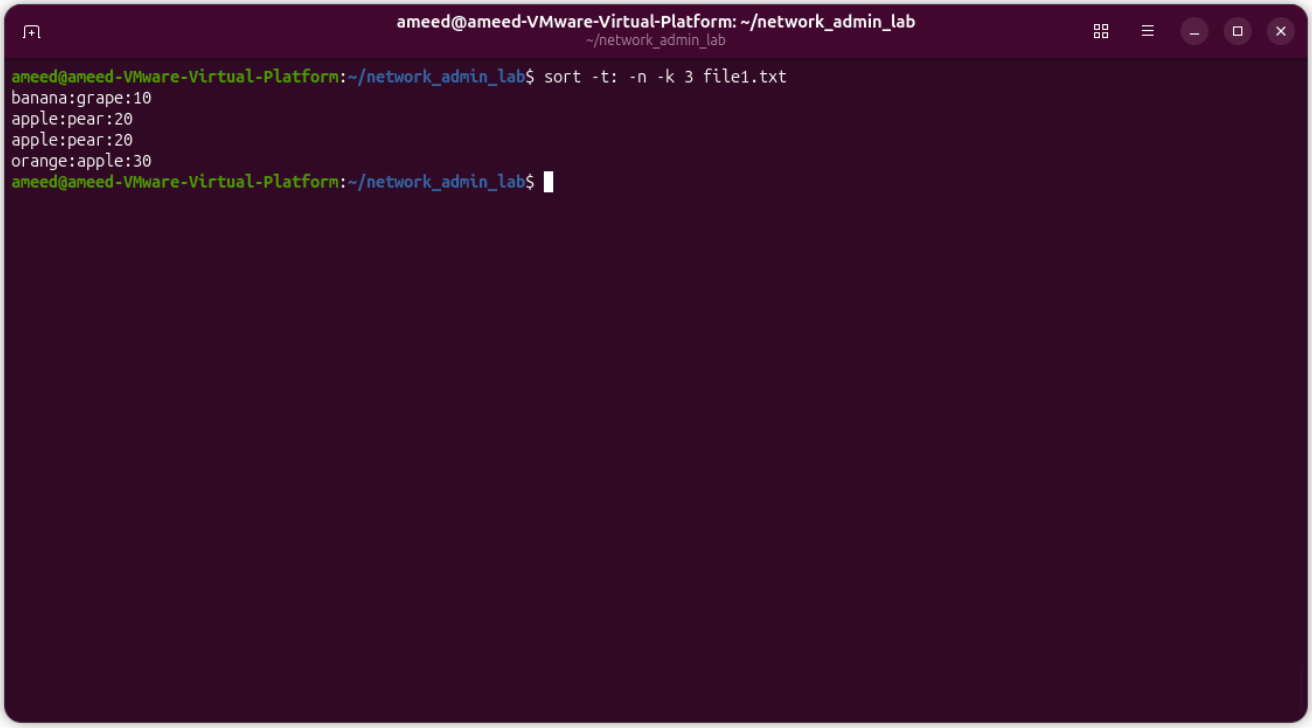
**Evidence 23** - 22-06-38 / sort file1.txt



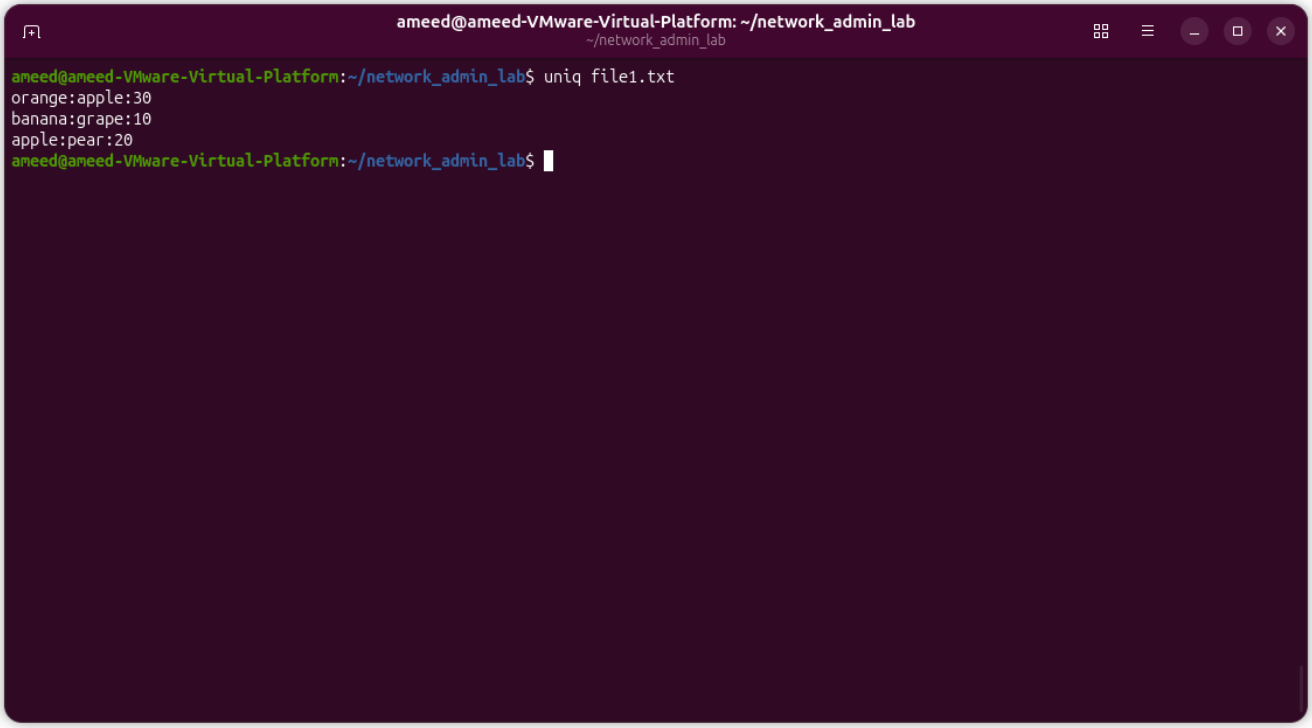

**Evidence 24** - 22-06-58 / sort -r file1.txt



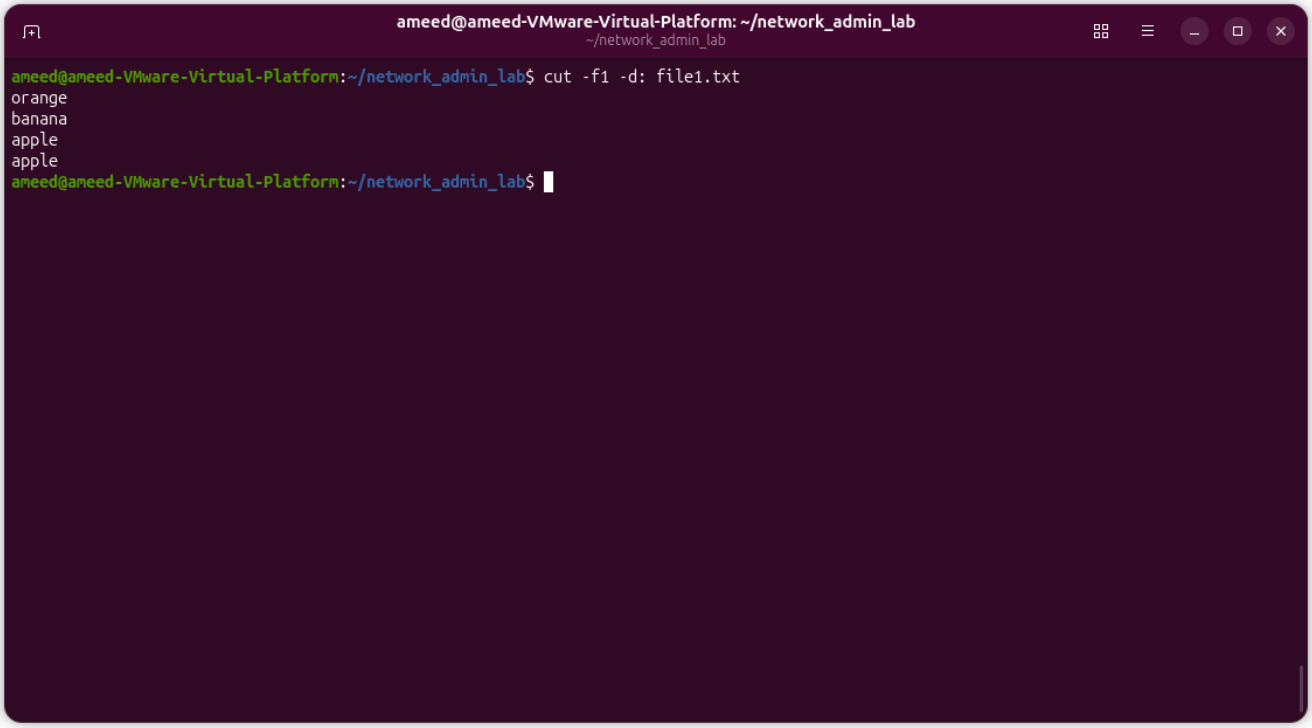
**Evidence 25** - 22-07-20 / sort -t: -n -k 3 file1.txt



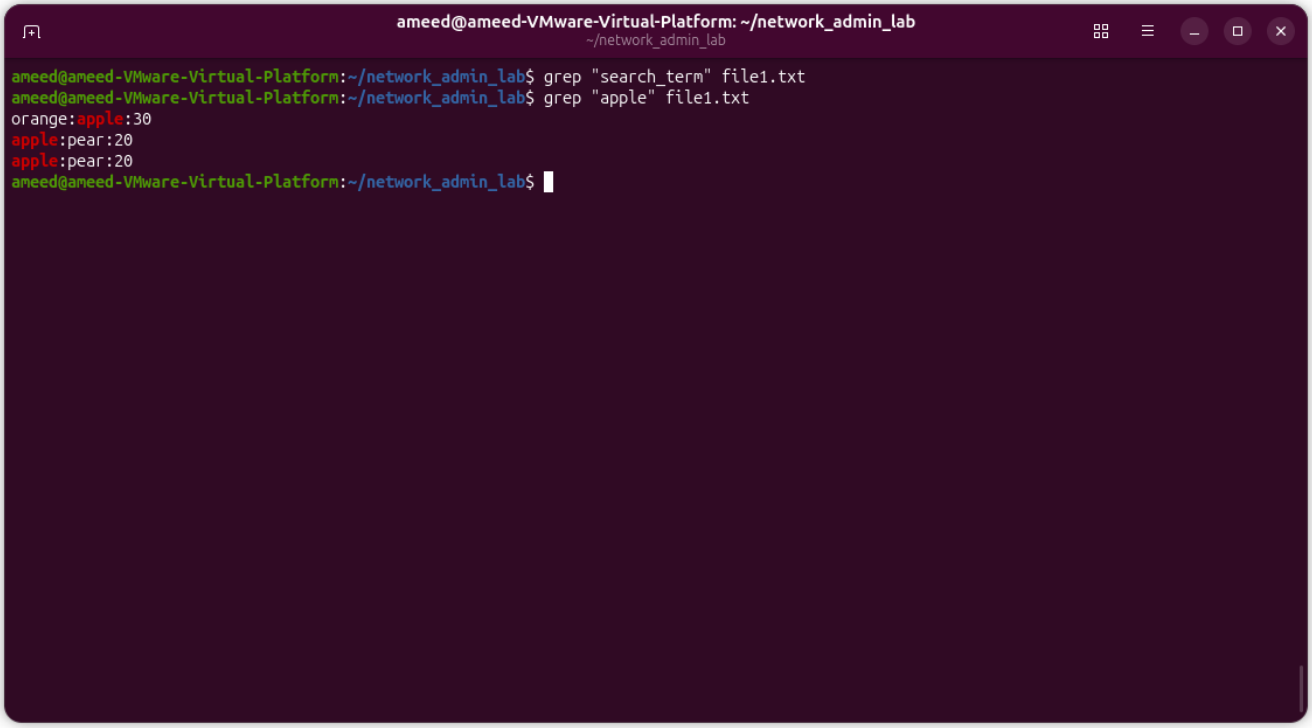

**Evidence 26** - 22-07-40 / uniq file1.txt



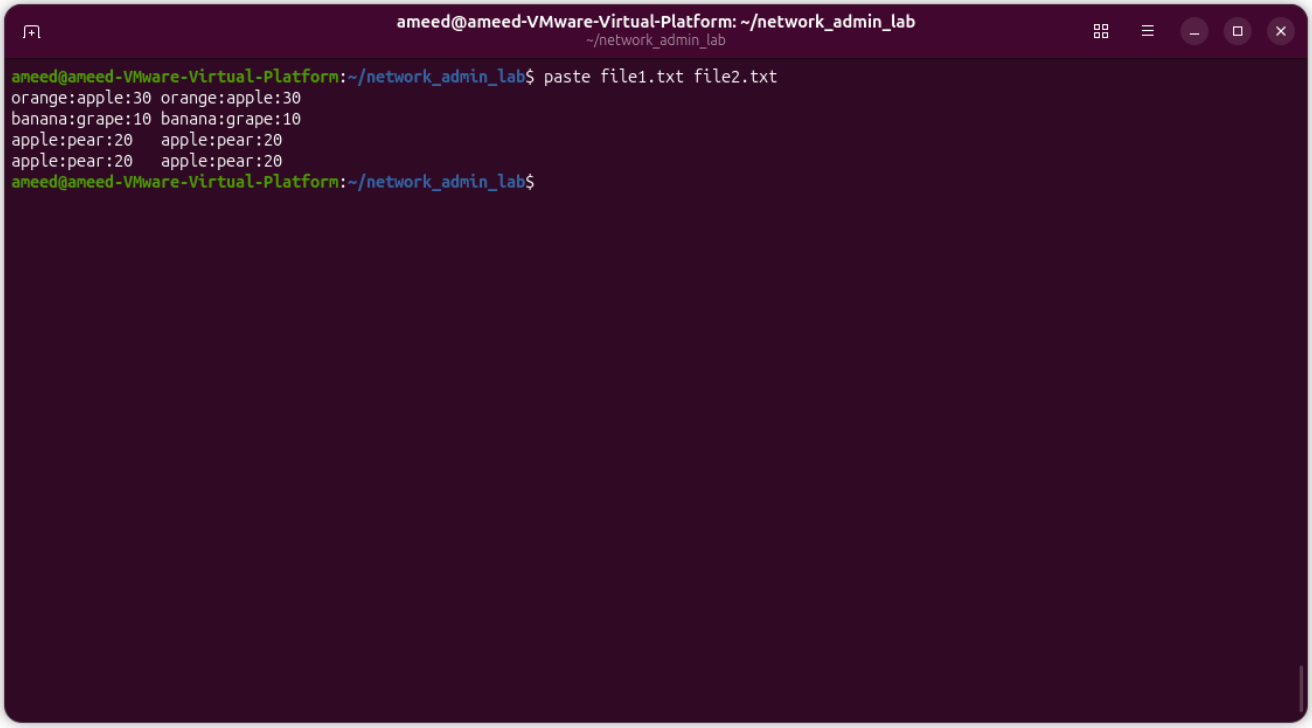
**Evidence 27** - 22-08-01 / cut -f1 -d: file1.txt



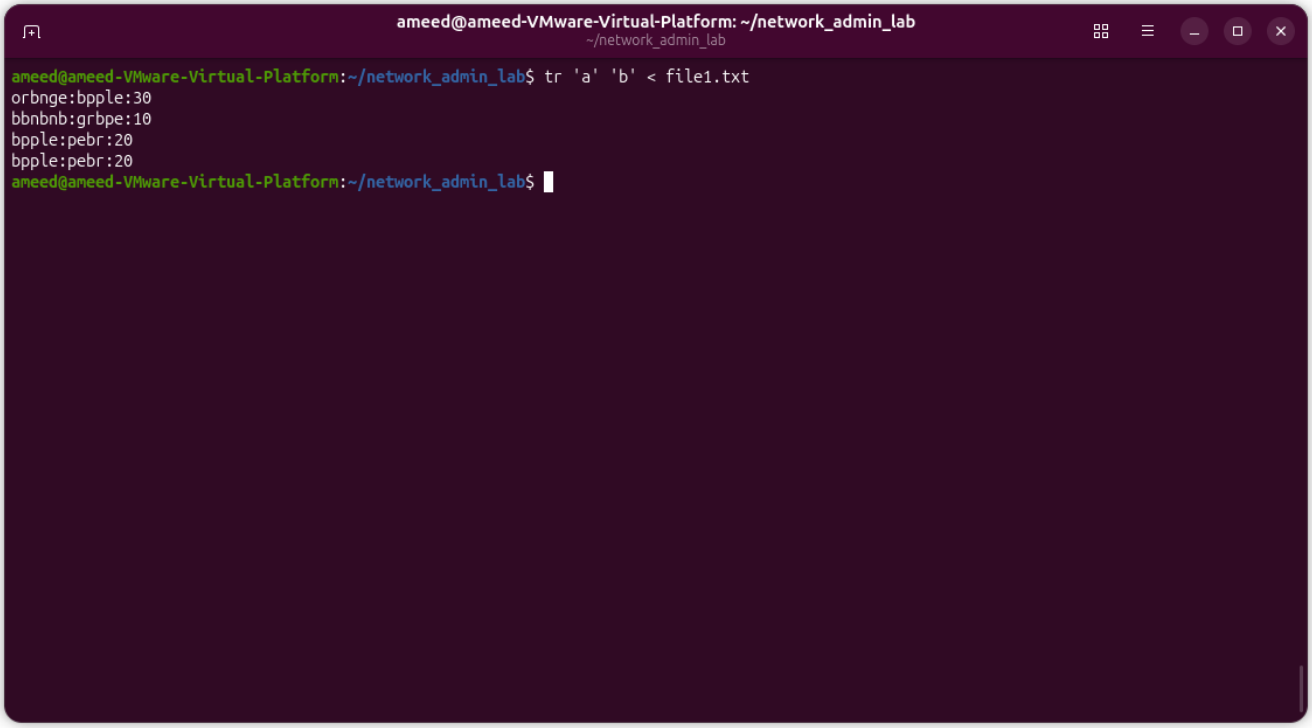

**Evidence 28** - 22-08-36 / grep search_term / apple



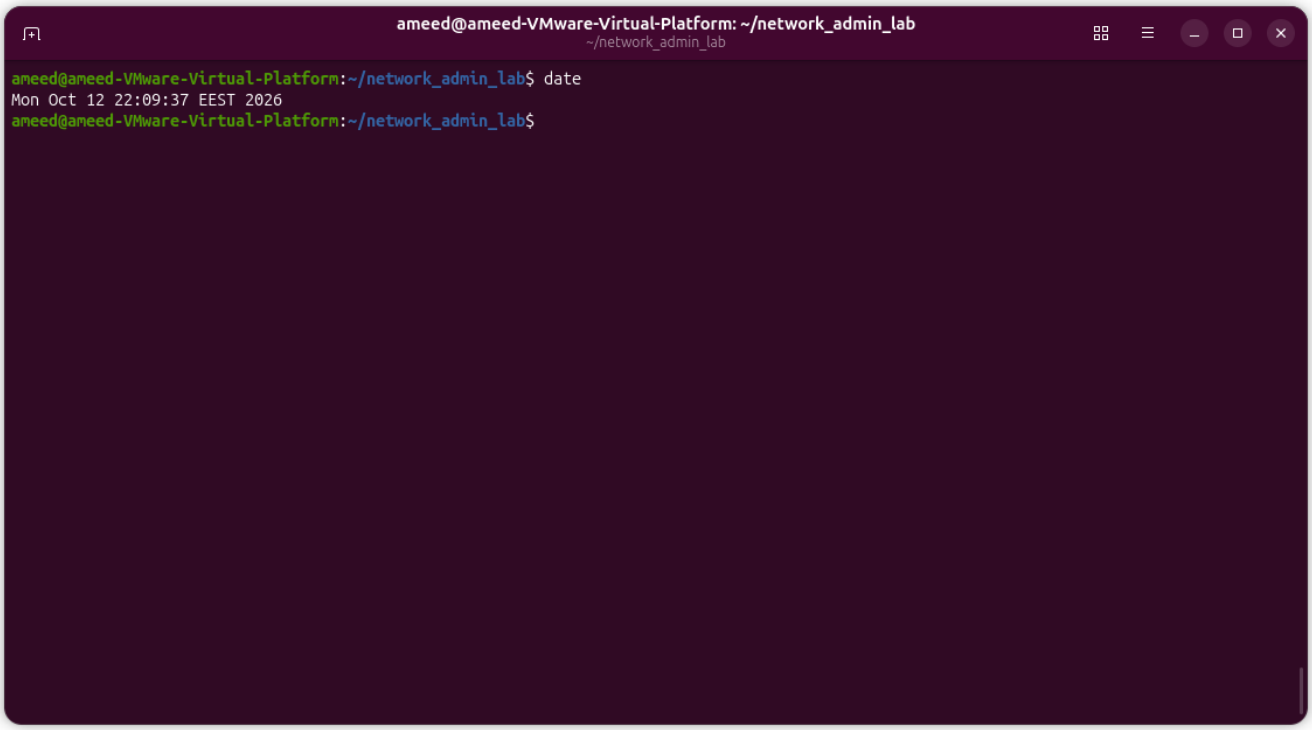
**Evidence 29** - 22-09-00 / paste file1.txt file2.txt



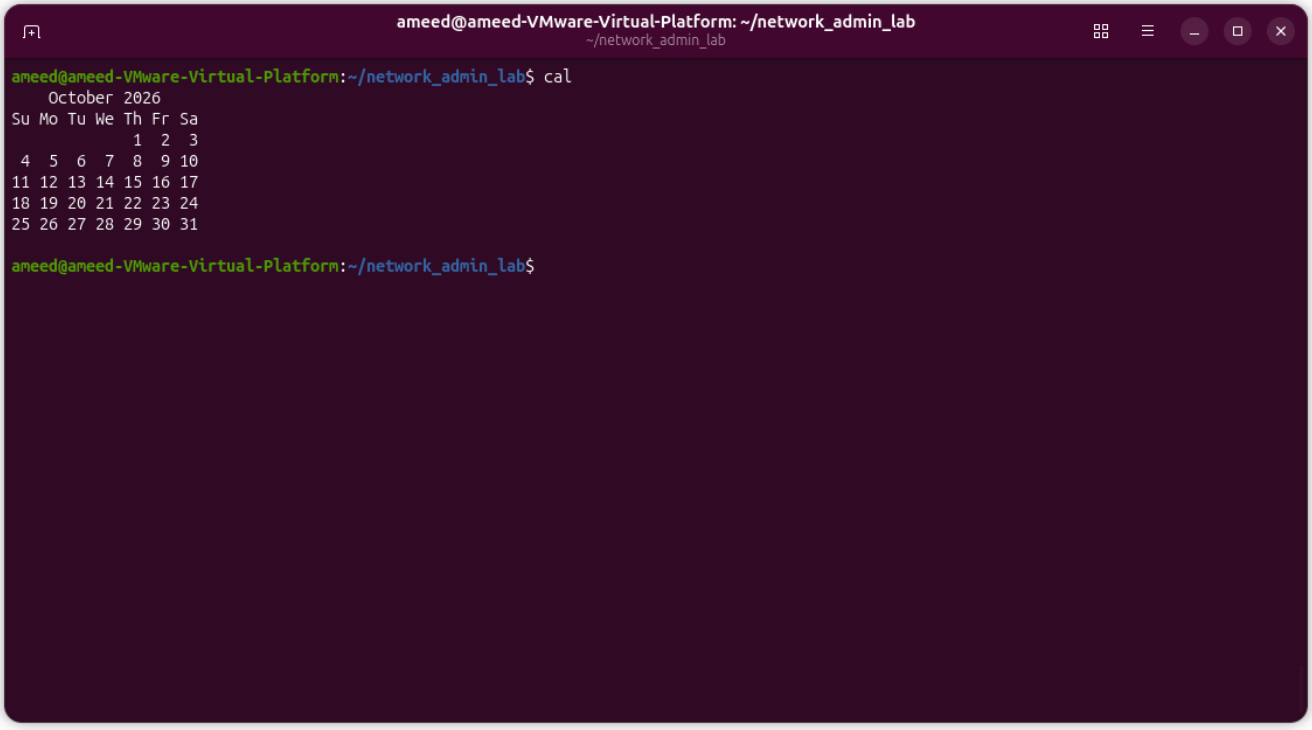

**Evidence 30** - 22-09-21 / tr 'a' 'b' < file1.txt



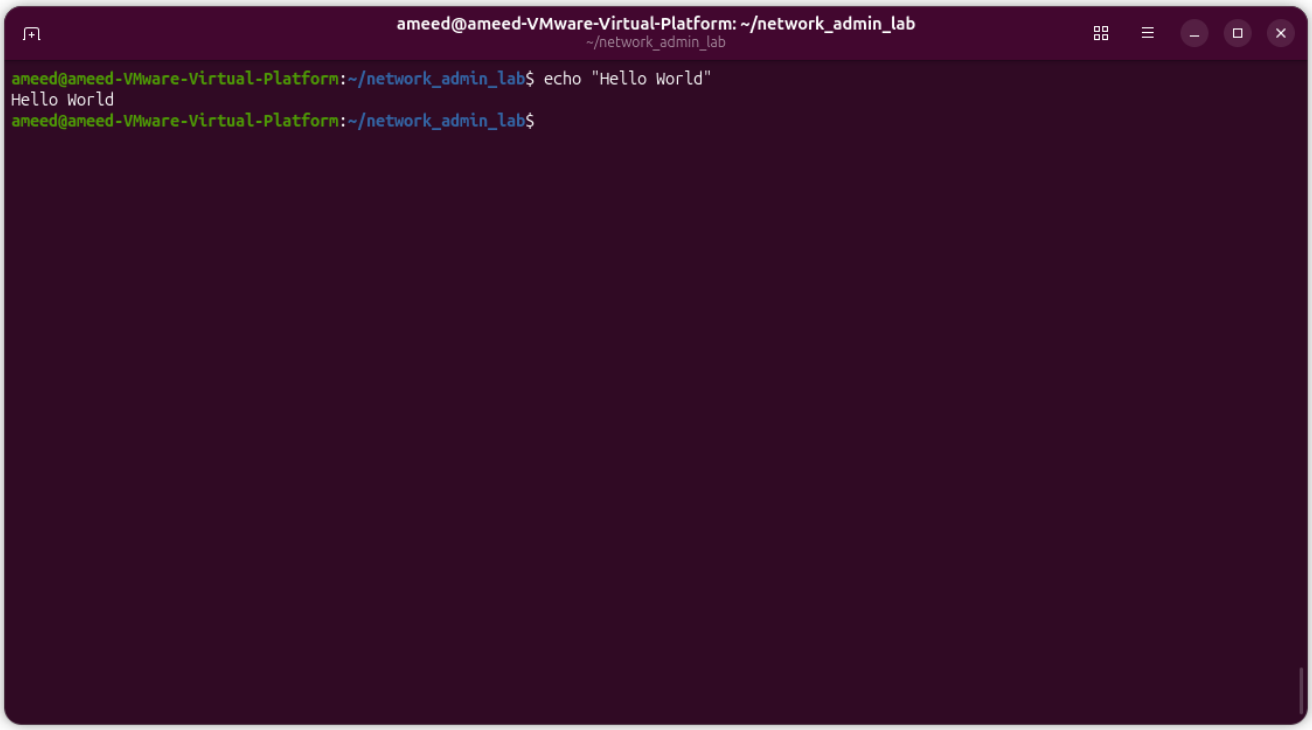
**Evidence 31** - 22-09-55 / date



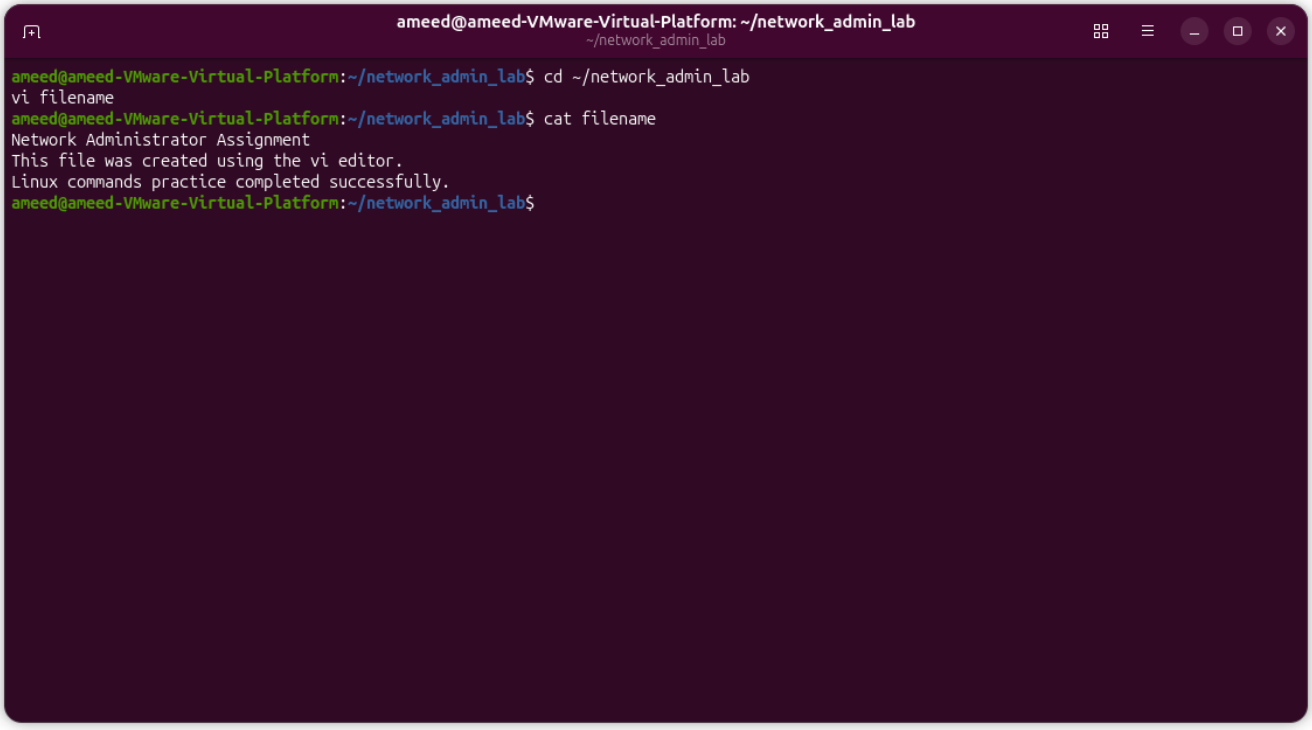

**Evidence 32** - 22-10-42 / cal



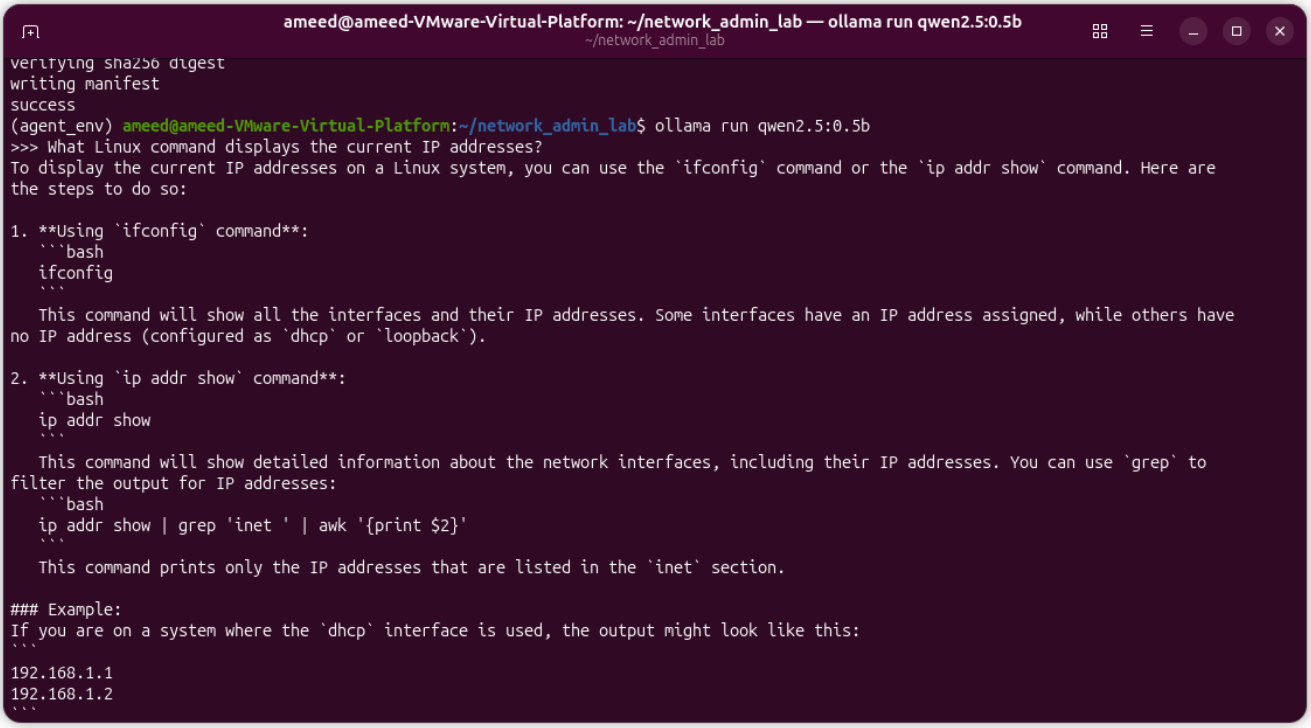
**Evidence 33** - 22-11-01 / echo Hello World



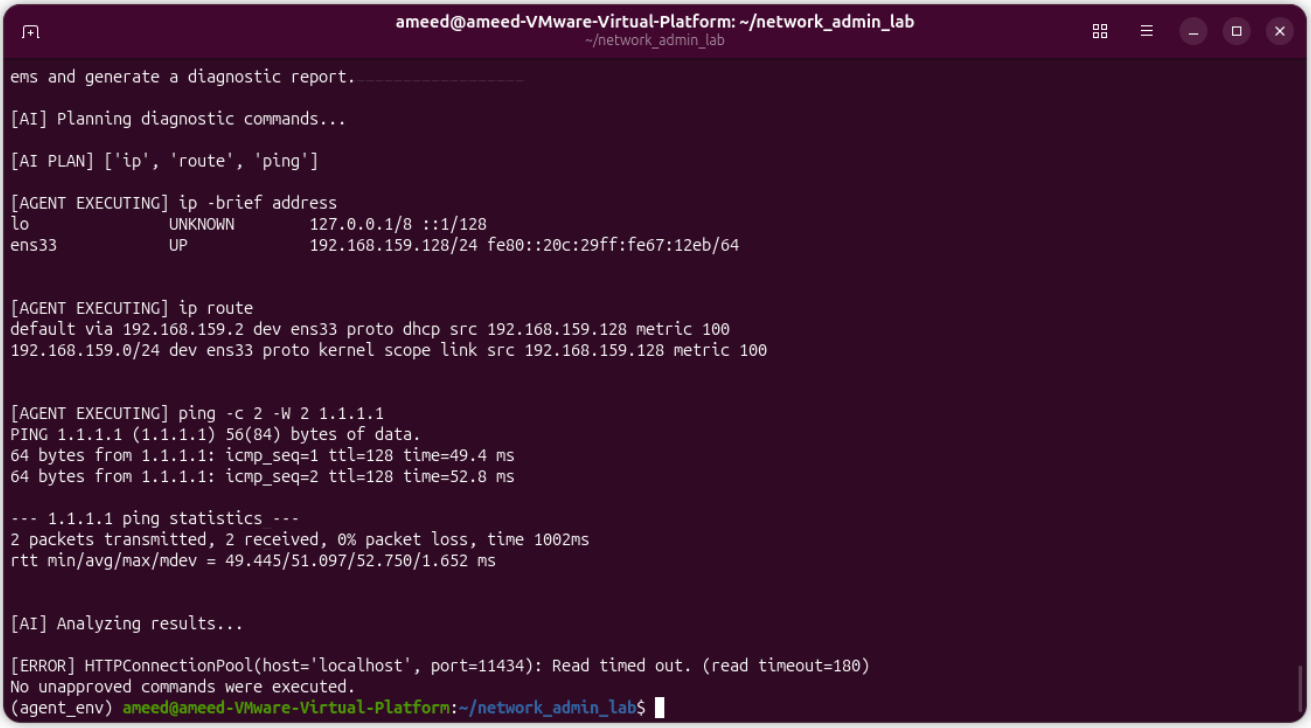

**Evidence 34** - 22-12-25 / vi filename



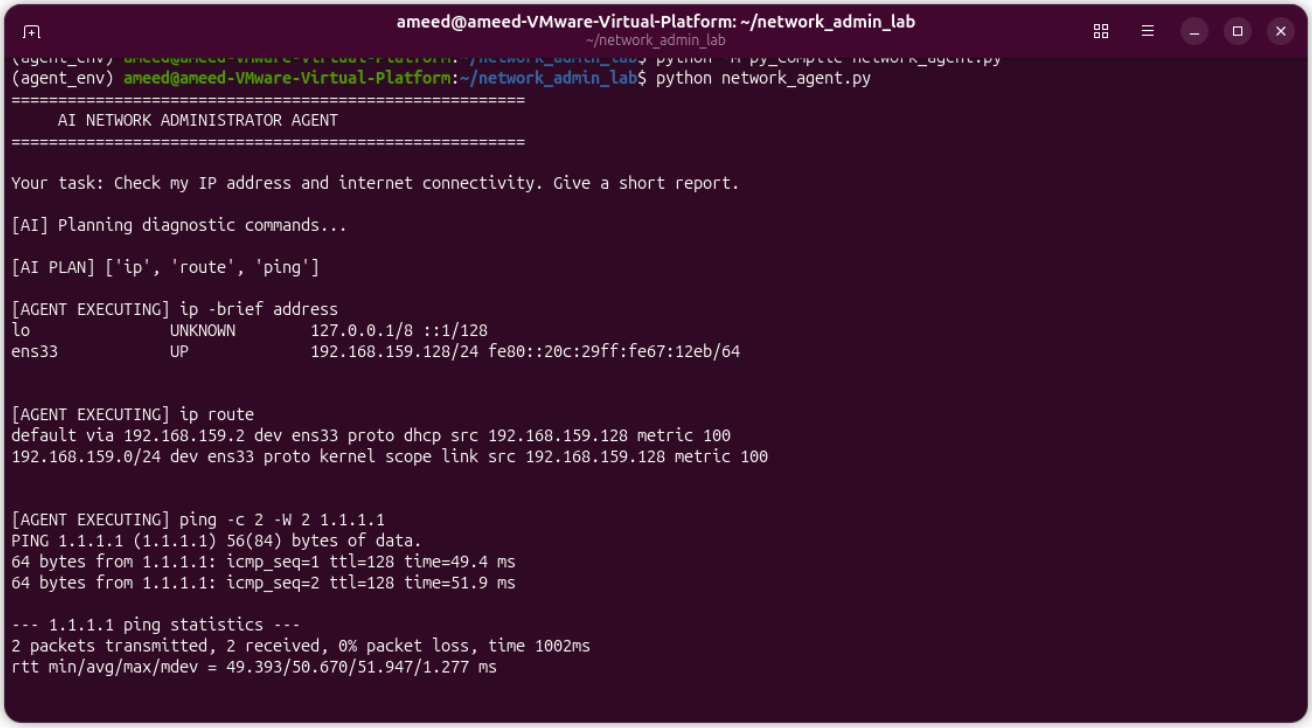
**Evidence 35** - 22-21-29 / Local Ollama model response



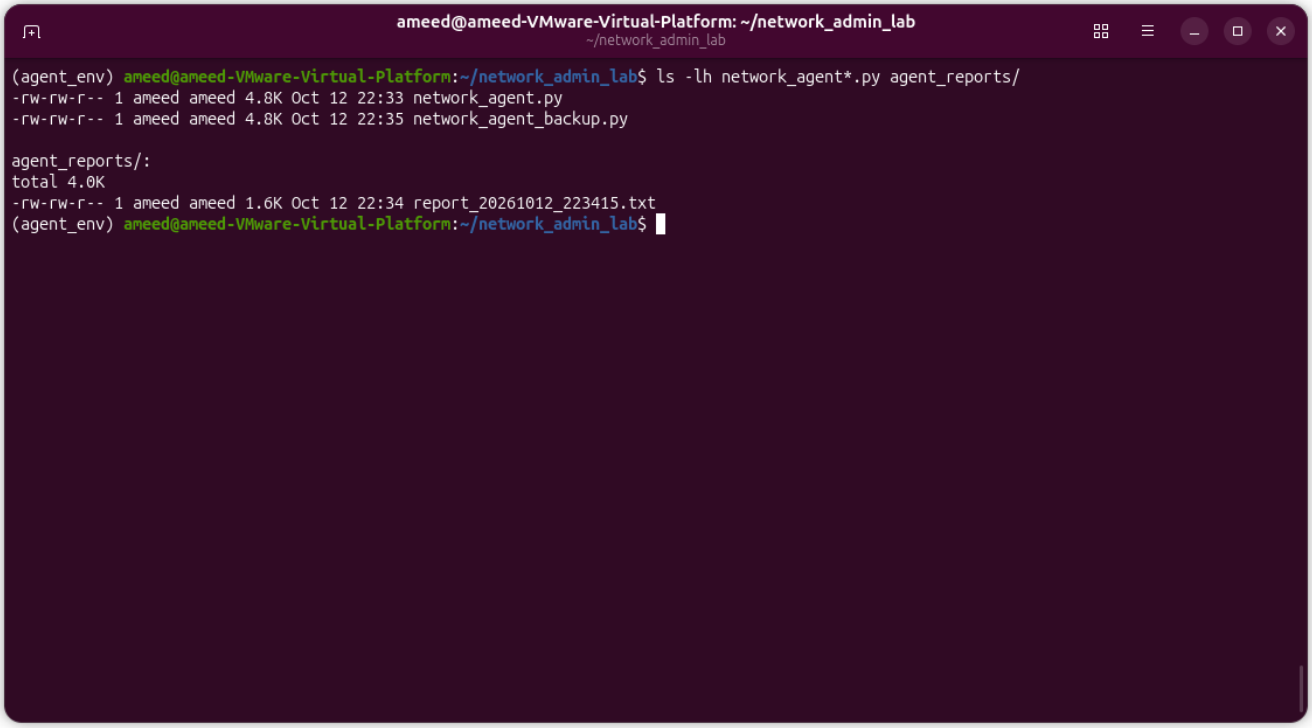

**Evidence 36** - 22-27-07 / Initial agent run and timeout



**Evidence 37** - 22-35-09 / Successful agent execution




**Evidence 38** - 22-35-57 / Source files and report verification



## Appendix B - Agent Implementation and Source Evidence

The submitted archive contains the complete Python source code and the original generated text report. The following is a summary of its central execution logic.

```
MODEL = "qwen2.5:0.5b"
API_URL = "http://localhost:11434/api/generate"

# AI chooses only keys from a fixed allowlist.
selected = plan.get("commands", [])
results[command_id] = execute_command(command_id)

# Commands run as argument arrays, never through a shell.
subprocess.run(command, capture_output=True,
               text=True, timeout=15, shell=False)
```

Evidence integrity: screenshots and output log reflect the files received from the user. AI-generated text is described as model output, not independently established fact.